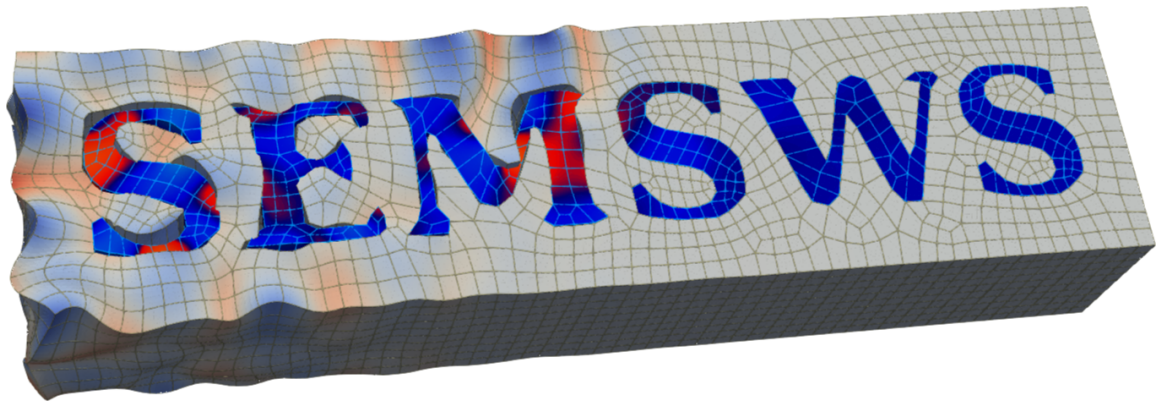




Spectral Element Method
Seismic Wave Simulator

ユーザーマニュアル

Version 0.1

2026 年 7 月 2 日

目次

第 1 章	はじめに	5
1.1	SEMSWS とは	5
1.2	対応する物理モデル	5
1.3	本マニュアルの構成	6
第 2 章	インストール	7
2.1	動作要件	8
2.2	Spack によるインストール (推奨)	9
2.3	手動ビルド	10
2.4	動作確認	14
2.5	GLVis (推奨)	15
第 3 章	クイックスタート	17
3.1	設定ファイル	17
3.2	シミュレーションの実行	21
3.3	結果の確認	22
第 4 章	入力ファイルリファレンス	25
4.1	YAML 形式の基礎	25
4.2	厳密キー検証 (Strict-Key Validation)	26
4.3	設定ファイルの全体構造	26
4.4	ルートレベルパラメータ	27
4.5	simulation セクション	27
4.6	vis セクション	28
4.7	mesh セクション	32
4.8	material セクション	35
4.9	boundary セクション	41

4.10	sources セクション	43
4.11	receivers セクション	47
4.12	device セクション	50
4.13	inversion セクション	50
4.14	HDF5 入出力スキーマと使い方	51
第 5 章	ツールとスクリプト	61
5.1	partition_mesh — メッシュの事前分割	61
5.2	combine_mesh — 分割メッシュの結合	62
5.3	evaluate_mesh — メッシュ品質の評価	63
5.4	refine_mesh — メッシュの均一細分化と Partitioning	64
5.5	ワークフロー例：メッシュ評価と細分化	66
5.6	可視化スクリプト	70
付録 A	サイト別ビルド設定	75
A.1	Earth Simulator (NVIDIA A100 GPU)	75
A.2	LUMI (AMD MI250X GPU)	78
開発チーム		81

第 1 章

はじめに

1.1 SEMSWS とは

SEMSWS (Spectral Element Method – Seismic Wave Simulator) は、スペクトル要素法 (SEM) に基づく地震波伝播シミュレーションソルバである。有限要素法ライブラリ MFEM^{*1}上に構築されている。MFEM は Lawrence Livermore National Laboratory を中心に開発されているオープンソースライブラリで、高次有限要素空間、多様なメッシュフォーマット、MPI 並列アセンブリ、CUDA/HIP/OpenMP による GPU/CPU ポータビリティなどを提供する。SEMSWS の中核機能の多くはこの MFEM の基盤に支えられており、単一のソースコードで CPU と GPU (CUDA/HIP) の双方で動作する。ここに MFEM 開発チームへの感謝を記す。

地震波動方程式の数値解法として広く用いられる SEM は、Gauss–Lobatto–Legendre (GLL) 点に基づく高次 Lagrange 補間を用いる手法であり、質量行列の対角化による効率的な陽的時間積分、非構造メッシュへの対応、弱形式による自由表面境界条件の自然な取り込み、そしてスペクトル収束に基づく高精度かつ低分散な解が得られるという特長を持つ。

1.2 対応する物理モデル

SEMSWS は以下の波動方程式を 2 次元・3 次元でサポートする：

- 等方性弾性波動方程式 (Isotropic Elastic)
- 等方性音響波動方程式 (Isotropic Acoustic)
- 粘弾性・粘音響 (Viscoelastic / Viscoacoustic)
- 固体 – 液体連成 (Fluid–Solid Coupling)

今後のアップデートで VTI や Full Anisotropy などのモデルへの対応も予定している。

^{*1} <https://mfem.org/>

1.3 本マニュアルの構成

第 2 章：インストール 依存ライブラリ (hypr, MFEM) の準備からビルドまでの手順を説明する。

各計算環境 (ローカル、ES、LUMI) でのビルド設定例も掲載。

第 3 章：クイックスタート 2D 音響波シミュレーションを例に、設定ファイルの記述から実行、結果の可視化までの一連の流れを説明する。

第 4 章：入力ファイルリファレンス config.yaml の全オプションを網羅的に記述する。

第 5 章：ツールとスクリプト メッシュの分割・細分化・品質評価ツールと可視化スクリプトの使い方

第 2 章

インストール

本章では SEMSWS 本体のビルド方法を 2 通り示す。

- §2.2 : Spack による自動インストール (推奨)
- §2.3 : 手動ビルド (依存ライブラリも自分で構築する場合)

依存ライブラリの一覧は §2.1、ビルド後の動作確認は §2.4 にまとめている。Earth Simulator や LUMI など HPC サイト固有の設定は付録 A を参照。

2.1 動作要件

表 2.1 依存ライブラリ

ソフトウェア	必須/任意	備考
C++17 コンパイラ	必須	-std=c++17 サポート
CMake ≥ 3.20	必須	ビルドシステム
MPI	必須	並列計算
MFEM ≥ 4.9	必須	MPI 有効でビルドすること。4.9 で導入された <code>mfem::Reshape・Dual<></code> 自動微分型に依存
hypr	必須	MFEM 経由でリンク
ADIOS2	必須	並列 I/O (BP 形式)
HDF5	必須	波形・受振器の I/O。SEMSWS が直接リンクする。MFEM と同じ HDF5 を使うこと (自動検出されない場合は <code>HDF5_ROOT</code> を指定)
yaml-cpp $\geq 0.9.0$	必須	設定ファイル解析。0.7/0.8 は <code><cstdint></code> 修正を欠きビルド不可 (自動取得されないため事前に用意する)
METIS	必須	メッシュ分割
NetCDF	任意	Cubit/Exodus メッシュの読み込み。MFEM 経由でリンク (<code>MFEM_USE_NETCDF</code>)
CUDA	任意	NVIDIA GPU サポート
HIP	任意	AMD GPU サポート
Umpire	任意	GPU メモリプール。性能への影響は小さく必須ではない (§2.3.6)

■pytest 用 Python パッケージ test/のテストスイートを実行する場合は、Python 3.10 以上に加えて以下をインストールする：

```
pip install pytest numpy h5py pyyaml matplotlib gmsh
```

matplotlib と gmsh は任意 (プロット系・流体-固体連成のメッシュ生成テスト用)。driver/以下の Python ドライバ semsws-driver は `pip install -e ./driver` で driver/pyproject.toml から依存が自動解決される。

2.2 Spack によるインストール (推奨)

SEMSWS リポジトリには `spack-repo/` 以下に Spack レシピが同梱されており、`hypr`・`MFEM`・`ADIOS2` などの依存ライブラリを含めて自動ビルドできる。Spack 1.0 以上が必須 (動作確認済みは `v1.1.x`・`v1.2.x`)。

■Step 1: SEMSWS を clone する

```
git clone https://github.com/SEMSWS/semsws.git
cd semsws
```

■Step 2: Spack を準備する spack コマンドが使えない場合は以下:

```
git clone --depth=1 https://github.com/spack/spack.git ~/spack
source ~/spack/share/spack/setup-env.sh
```

`source` 行は新しいシェルを開くたびに必要なので、`~/.bashrc` に追記しておくといよい。

■Step 3: SEMSWS の Spack repo を登録する

```
spack repo add ./spack-repo
```

■Step 4: cmake を external 登録する (推奨) 何もしないと Spack が cmake をソースからビルドして 10 分以上かかるので、システム cmake (3.20 以上) を external として宣言しておく:

```
mkdir -p ~/.spack
cat >> ~/.spack/packages.yaml <<'EOF'
packages:
  cmake:
    externals:
      - spec: cmake@3.28.0          # `cmake --version` に合わせる
        prefix: /usr               # `which cmake` で確認した prefix
        buildable: false
EOF
```

■Step 5: インストールしてロードする

```
spack install semsws@main
spack load semsws
which semsws      # /path/to/spack/opt/.../semsws-.../bin/semsws
```

最初のビルドは依存ライブラリを含めて 30-60 分かかる。

■サイト固有環境 (ES, LUMI) Earth Simulator や LUMI ではコンパイラのパッチや精度設定、GPU-aware MPI などサイト固有の調整が必要となる。`spack-repo/site-config/<site>/` にロー

ドスクリプトと `packages.yaml` テンプレートが用意されている。具体的な spec と手順は付録 A を参照。

2.3 手動ビルド

Spack を使わず依存ライブラリを自分で構築する場合の手順を示す。SEMSWS は `hypre`・`MFEM`・`yaml-cpp`・`ADIOS2`・`HDF5` を必須依存とし、いずれも自動取得されないため事前にビルドしておく (`hypre` → `MFEM` は GPU 精度設定を揃える必要があるためこの順、`yaml-cpp`・`ADIOS2`・`HDF5` は独立なので順不同)。最後に SEMSWs 本体をビルドする。Umpire は必須ではなく、付ける場合のみ §2.3.6 の手順を追加する。

注意：本節のコマンドはあくまで一例である。パス・コンパイラ・GPU アーキテクチャ・モジュール構成・各ライブラリのバージョンは環境ごとに異なるため、それぞれの環境に合わせて読み替えること。

■ `_DIR` と `_ROOT` の使い分け CMake に依存ライブラリの場所を渡すとき、`-DXxx_DIR` は `XxxConfig.cmake` が置かれたディレクトリ (CMake 設定ディレクトリ) を、`-DXxx_ROOT` はインストールプレフィックスを指す。依存をインストール済みなら `_ROOT` を使うのが簡便である。**例外：**MFEM 自身のビルドで `hypre` を指す場合は、MFEM が独自の `find` スクリプトを使うため `HYPRE_ROOT` が効かず、`HYPRE_DIR` に `hypre` のインストールディレクトリを渡す。

2.3.1 hypre

`hypre` は `src/`以下で CMake ビルドする。CPU (単精度) の最小構成は：

```
cd hypre/src
cmake -B build \
  -DHYPRE_ENABLE_SINGLE=ON -DHYPRE_ENABLE_MPI=ON \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

倍精度の場合は `HYPRE_ENABLE_SINGLE=ON` を省く。GPU ビルドではバックエンドに応じて表 2.2 のフラグを追加する。

表 2.2 hypre CMake フラグ（バックエンド別の差分）

バックエンド	追加フラグ
CPU	(なし)
CUDA	-DHYPRE_ENABLE_MIXEDINT=ON -DHYPRE_ENABLE_UNIFIED_MEMORY=ON -DHYPRE_ENABLE_CUDA=ON -DHYPRE_ENABLE_GPU_AWARE_MPI=ON - DCMAKE_CUDA_ARCHITECTURES=80 -DHYPRE_ENABLE_CUSOLVER=OFF
HIP	-DHYPRE_ENABLE_MIXEDINT=ON -DHYPRE_ENABLE_UNIFIED_MEMORY=ON -DHYPRE_ENABLE_HIP=ON -DHYPRE_ENABLE_GPU_AWARE_MPI=ON - DCMAKE_HIP_ARCHITECTURES=gfx90a

HYPRE_ENABLE_CUSOLVER は SEMSWs では不要なので OFF でよい。cuSOLVER のリンクは古い GCC (8 系など) で `extern "C"` 関連のコンパイルエラーを引き起こすため、無効化しておくトラブルを避けられる。SEMSWS 向けには hypre 2.33.0 が最も安定して動作する。Umpire なしで GPU ビルドすると、hypre が警告を出すことがあるが、無視して差し支えない。(Umpire を併用する場合は §2.3.6 を参照。)

2.3.2 MFEM

SEMSWS は MFEM 4.9 以上を要求する。CMake でビルドする場合の例を示す：

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=/path/to/hyre/install
cmake --build build -j$(nproc)
cmake --install build
```

HYPRE_DIR には hypre のインストールディレクトリを渡す (前述の規約の例外)。NetCDF は必須でないが、CUBIT など生成したメッシュを読み込むには必要。GPU ビルドでは -DMFEM_USE_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=80 (NVIDIA) あるいは -DMFEM_USE_HIP=ON -DAMDGPU_TARGETS=gfx90a (AMD) と CMAKE_CXX_COMPILER を切り替える。複数の GPU デバイスを持つクラスターでは CMAKE_CUDA_ARCHITECTURES ではうまくいかないことがあり、その場合は -DCUDA_ARCH=sm_80 で明示する。

■精度の一貫性 hydre の HYPRE_ENABLE_SINGLE と MFEM の MFEM_PRECISION=single は必ず一致させる。SEMSWS は MFEM の精度設定を自動的に引き継ぐため、hydre と MFEM を揃えておけば SEMSWs 側での再指定は不要。

2.3.3 yaml-cpp

設定ファイルの解析に使う。0.9.0 以上が必須 (0.7/0.8 は不可)。該当タグを取得してビルドする：

```
git clone -b 0.9.0 https://github.com/jbeder/yaml-cpp.git
cd yaml-cpp
cmake -B build -DYAML_BUILD_SHARED_LIBS=ON \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

2.3.4 ADIOS2

FWI のカーネル・モデル I/O (BP5 形式) に使う。MPI 有効でビルドする：

```
cmake -B build -DADIOS2_USE_MPI=ON \
  -DADIOS2_USE_Python=OFF -DBUILD_TESTING=OFF \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

2.3.5 SEMSWs

依存ライブラリをすべてインストールしたら、前述の規約に従い _ROOT で場所を渡してビルドする：

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DMFEM_ROOT=/path/to/mfem/install \
  -Dyaml-cpp_ROOT=/path/to/yaml-cpp/install \
  -DAdios2_ROOT=/path/to/ADIOS2/install \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
cmake --install build
```

hypré・METIS・NetCDF は MFEM 経由で推移的にリンクされるため、SEMSWS 側では指定不要である。HDF5 のみ SEMSWs が直接 `find_package` するが、通常はシステムにインストール済みの HDF5 が自動検出される。見つからない場合は `-DHDF5_ROOT=` に MFEM と同じ HDF5 の prefix を渡す。

表 2.3 主要な CMake オプション

オプション	説明
MFEM_DIR / MFEM_ROOT	MFEM の場所(必須)。 <code>_DIR</code> は CMake 設定ディレクトリ、 <code>_ROOT</code> はインストールプレフィックス (前述の規約)
adios2_DIR / adios2_ROOT	ADIOS2 の場所 (同上)
CMAKE_BUILD_TYPE	Release (推奨) または Debug
CMAKE_INSTALL_PREFIX	<code>cmake --install</code> での配置先
CMAKE_CUDA_ARCHITECTURES	NVIDIA GPU 向け (例: 80)
AMDGPU_TARGETS	AMD GPU 向け (例: gfx90a)
SEM_FORCE_CPU	MFEM が GPU 対応でも CPU 専用にビルド (既定 OFF)
SEM_USE_GPU_AWARE_MPI	GPU-aware MPI があることをビルド時に表明 (既定 OFF)
SEM_ENABLE_PROFILING	領域別タイミング計測を有効化 (MFEM_DEVICE_SYNC のオーバーヘッド付き。既定 OFF)

`SEM_USE_GPU_AWARE_MPI=ON` は LUMI Cray MPICH (`MPICH_GPU_SUPPORT_ENABLED=1`) や CUDA 対応 OpenMPI など GPU-aware が確認できる環境でのみ指定すること。

GPU ビルドは MFEM が CUDA または HIP 付きでビルドされていれば SEMSWs は自動的に GPU ビルドとなる。CPU 専用にした場合は `SEM_FORCE_CPU=ON` を指定する。

2.3.6 (任意) Umpire を併用する場合

Umpire は GPU 上のメモリプール管理を担うライブラリだが、SEMSWS の性能への影響は小さく必須ではない。CPU ビルドでは不要で、GPU ビルドでも省略してよい。メモリ断片化を明示的なプールで抑えたい場合のみ、以下を追加する。

まず Umpire をビルドする：

```
cmake -B build \
  -DENABLE_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DENABLE_MPI=ON -DUMPIRE_ENABLE_C=ON \
  -DCMAKE_INSTALL_PREFIX=$(pwd)/install
cmake --build build -j$(nproc)
```

```
cmake --install build
```

そのうえで各ライブラリの cmake 行に以下を追加する：

- hypr : `-DHYPRE_ENABLE_UMPIRE=ON -Dumfire_ROOT=/path/to/umfire/install`
- MFEM : `-DMFEM_USE_UMPIRE=ON -DUMPIRE_ROOT=/path/to/umfire/install`
- SEMSW : `-Dumfire_ROOT=/path/to/umfire/install`

hypr と MFEM を Umpire 付きでビルドしたら、SEMSW も必ず Umpire 付きでビルドして構成を一貫させる。

2.4 動作確認

ビルドに成功すると `build/src/` 以下に実行ファイル群が生成される。中核となるのは表 2.4 のシミュレーション本体とメッシュ・可視化ツールである。

表 2.4 生成される主要な実行ファイル

実行ファイル	説明
<code>semsw</code>	シミュレーション本体
<code>partition_mesh</code>	メッシュの事前分割
<code>refine_mesh</code>	メッシュの細分化
<code>evaluate_mesh</code>	メッシュ品質の評価
<code>combine_mesh</code>	分割メッシュの結合（可視化用）
<code>sem_viz</code>	ADIOS2 BP（材料・カーネル場）を GLVis/ParaView/GMT 形式に変換

上記に加え、FWI 用のカーネル処理・モデル更新ツール群（`smooth_kernel`、`sum_kernels`、`mask`、`precondition_gradient`、`clip_field`、`scale_field`、`lbfgs_direction`、`update_model`、`max_abs`、`semsw_export_model`）も生成される。これらの使い方は第 5 章で扱う。

```
./build/src/semsw --help
```

■**pytest による検証** `test/` には用途別のテストが用意されている。`--build-dir` には実行ファイル群のある場所を渡す。ローカルビルドではビルドディレクトリ（`./build`）、`cmake --install` や Spack で導入した場合はインストールプレフィックスを指定する。

```
# 全テスト（CPU、シリアル）
pytest test/ --build-dir ./build -v
```

```
# MPI並列・GPUデバイス指定
pytest test/ --build-dir ./build --np 4 --device cuda -v

# SLURMクラスタ (srunでランチャを差し替え。--build-dirはinstall先)
pytest test/ --build-dir <install_dir> --device cuda \
    --mpi-cmd=srun --np 2 -s -v -x

# 波形検証のみ・出力形式整合性のみ・流体--固体連成のみ
pytest test/waveform/ --build-dir ./build -v
pytest test/consistency/ --build-dir ./build -v
pytest test/coupling/ --build-dir ./build --np 4 -v
```

波形検証テスト (test/waveform/) はシミュレーション結果をリファレンス波形と比較し、正規化 L2 ノルムが閾値以内であることを検証する。ケース名に `analytic` を含むものは解析解との比較、それ以外はすべて SPEC-FEM との比較である。--keep-results を付けると各テストディレクトリの results/ にシミュレーション結果が保存され、同ディレクトリ内の ref/ と直接比較できる。

出力形式整合性テスト (test/consistency/) は受振器の出力形式 (ASCII、HDF5、SU) 間で波形が数値的に一致することを検証する。**連成テスト** (test/coupling/) は流体－固体連成問題を検証する。

Spack でインストールした場合は--build-dir に `spack find --paths semsws` で得た prefix を渡す：

```
SEMSWS_PREFIX=$(spack find --paths semsws | awk 'NR>1 && /\// {print $NF; exit}')
pytest test/consistency/ --build-dir $SEMSWS_PREFIX --np 4 -v
```

2.5 GLVis (推奨)

GLVis は MFEM プロジェクトが公開している軽量な可視化ツールである^{*1}。SEMSWS が results/wavefield/ に出力する MFEM 形式のメッシュとグリッド関数を読み込み、波動場をフレーム単位で確認できる。SEMSWS 自体の動作には必須ではないが、第 3 章の Quick Start チュートリアルが格段に確認しやすくなる。(ADIOS2 BP 形式で出力された材料場や FWI カーネルを可視化する場合は、表 2.4 の sem_viz で GLVis/ParaView/GMT 形式に変換する。詳細は第 5 章を参照。)

^{*1} <https://glvis.org/>

■ビルド 前節でインストールした MFEM をそのまま参照できる。Debian / Ubuntu ではあらかじめシステムパッケージを入れておく：

```
sudo apt install libsdl2-dev libglew-dev libfontconfig-dev libfreetype-dev

git clone https://github.com/GLVis/glvis.git
cd glvis
make -j MFEM_DIR=../mfem
```

■基本的な使い方

メッシュのみ

```
glvis -m mesh.mesh
```

メッシュ + 1フレームの波動場

```
glvis -m results/wavefield/mesh.mesh -g results/wavefield/sol_000500.gf
```

分割 (MPI 並列) メッシュ — N は出力ランク数

```
glvis -np 2 -m results/wavefield/mesh -g results/wavefield/sol_000500
```

分割メッシュ形式では GLVis が mesh.000000、mesh.000001、... と対応する sol_000500.000000... を自動的に読み込み、1つのウィンドウに再結合して表示する。キーバインドは GLVis の README^{*2} を参照。

^{*2} <https://github.com/GLVis/glvis>

第 3 章

クイックスタート

この章では、2次元音響波シミュレーションを例に、設定ファイルの作成からシミュレーションの実行、結果の可視化に至る一連の流れを説明する。使用する例題は `examples/visualization/2D/acoustic/` に含まれている。

3.1 設定ファイル

SEMSWS のシミュレーションは、すべてのパラメータを 1 つの YAML ファイル (`config.yaml`) で記述する。以下、本例題の設定ファイルをセクションごとに説明する。

3.1.1 シミュレーション基本設定

```
name: Visualization Test - 2D Acoustic

simulation:
  dimension: 2
  order: 4
  steps: 10000
  dt: 0.0001
  cfl_factor: 0.3
  directory: ./results
  log_interval: 1000
  summary_file: summary.txt
```

`dimension` は空間次元 (2 または 3)、`order` は多項式次数 (スペクトル要素法の次数) を指定する。`steps · dt` で時間ステップ数と時間刻み幅 Δt を設定する。本例題では $\Delta t = 0.0001 \text{ s}$ で 10000 ステップ、すなわち 1 秒間のシミュレーションとなる。`cfl_factor` は CFL 安定条件の安全係数、`directory` は出力先ディレクトリ、`log_interval` は進捗ログの間隔、`summary_file` はサマリー

の出力ファイル名である。冒頭の `name` はシミュレーションの識別名である（任意）。

3.1.2 メッシュ

```
mesh:
  type: internal
  origin: [0.0, 0.0]
  size: [1000.0, 500.0]
  elements: [50, 25]
  max_freq: 75.0
  ppw: 5.0
```

`type: internal` は SEMSWS 内部で regular/structured メッシュを自動生成する設定である。`size` でドメインサイズ (m)、`elements` で要素数を指定する。本例題では 1000 m × 500 m の領域を 50 × 25 要素で分割する。メッシュの可視化出力は後述の `vis: mesh: enabled: true` で行う。`max_freq` と `ppw` はメッシュ品質評価用のパラメータで、それぞれ最大周波数と 1 波長あたりのサンプリング数 (points per wavelength) を表す。これらは `summary.txt` の出力や、後述する `evaluate_mesh` プログラムで使用される。

外部メッシュ (GMSH、CUBIT/ExodusII 等) を使用する場合は `type: external` とし、`file:` にファイルパスを指定する：

```
mesh:
  type: external
  file: "model.exo"
```

3.1.3 物性

```
material:
  type: isotropic_acoustic
  format: constant
  vp: 1500.0
  rho: 1000.0
  attenuation:
    enabled: false
```

均質な音響媒質を定義している。`type` で物理モデル (`isotropic_acoustic`) を選択し、`format: constant` で均質モデルであることを指定する。P 波速度 1500 m/s、密度 1000 kg/m³ の設定である。

3.1.4 境界条件

```
boundary:
  absorbing:
    sides: []
    thickness: 100.0
    alpha: 0.008

  dirichlet: [top, right, left, bottom]
```

`dirichlet` で全辺に Dirichlet (固定端) 境界条件を設定している。吸収境界 (Cerjan スポンジ層) は `sides: []` で無効化されている。吸収境界を有効にする場合は `sides: [top, right, left, bottom]` のように適用面を指定する。海域での反射法地震探査や屈折法地震探査のシミュレーションでは、海面を Dirichlet に設定する必要がある (`dirichlet: [top]`)。

3.1.5 震源

```
sources:
  shot_id: 1
  list:
    - type: pressure
      location: [500.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

ドメイン中央付近 (500 m, 400 m) に 30 Hz の Ricker ウェーブレットを持つ圧力点震源を 1 つ配置している。`delay` はウェーブレットのピーク到達時刻 (秒) である。`shot_id` はこのショットの ID である。各震源には任意で `id・name` を付けられる。

3.1.6 受振器

```
receivers:
  output:
    formats: [ascii]
    filename: seismograms
    type: [PS]

  list:
```

```

- name: "R001"
  location: [400.0, 300.0]
- name: "R002"
  location: [500.0, 300.0]
- name: "R003"
  location: [600.0, 300.0]

```

3 点の受振器を配置し、圧力スカラー場 (PS) を記録する。`formats` は出力形式のリストで、`ascii`・`hdf5`・`su` を指定できる (`su` は Seismic Unix 形式)。`ascii` では、1 列目が時刻、2 列目が振幅のテキストファイルが出力される。`filename` は HDF5/SU 出力のベース名で、ASCII は受振器ごとに `R001_0001.p` のように命名される。

3.1.7 可視化出力 (vis)

波動場・物性・メッシュの可視化出力はトップレベルの `vis`: セクションでまとめて設定する。

```

vis:
  wavefield:
    enabled: true
    interval: 1000
    fields: ["PS"]
    components: [-1]
    formats:
      - glvis
      - type: paraview
        refinement: 4
        data_format: binary32
      - type: gmt
        resolution: [100, 50]
  material:
    enabled: true
    fields: [vp, rho]
    formats:
      - glvis
  mesh:
    enabled: true

```

`vis: wavefield` で 1000 ステップごとに波動場スナップショットを出力する。`formats` には複数の形式を同時に指定でき、オプション不要の `glvis` は文字列のみで指定し、ParaView・GMT は `type`: 付きのマップで追加設定を渡す:

- **GLVis** — MFEM ネイティブ形式。glvis コマンドで対話的に可視化
- **ParaView** — VTU 形式。ParaView で 3 次元可視化に適する
- **GMT** — テキストグリッド形式 (x y value)。resolution でグリッド解像度を指定

vis: material は物性分布 (vp・rho など)、vis: mesh: enabled: true はメッシュを出力する。
components: [-1] はベクトル場の全成分を表す (スカラー場では無視される)。

3.1.8 デバイス設定

```
device:  
  type: cpu
```

cpu、cuda、hip から選択する。GPU ビルドでも cpu を指定すれば CPU 上で実行される (ただし動作はかなり遅いため、CPU 実行時は CPU ビルドの使用を推奨する)。

3.2 シミュレーションの実行

設定ファイルのあるディレクトリで以下のコマンドを実行する:

```
cd examples/visualization/2D/acoustic  
  
# 4プロセスで実行  
mpirun -np 4 ../../../../../../build/src/semsws \  
  --config config.yaml
```

同ディレクトリに含まれる run.sh を使うこともできる:

```
bash run.sh 4      # 引数はMPIプロセス数
```

実行が完了すると、results/ディレクトリに以下が生成される:

表 3.1 出力ファイル一覧

パス	説明
summary.txt	シミュレーション情報（物性、メッシュ、性能等）
R001_0001.p 等	受振器波形（ASCII テキスト）
wavefield/gmt/	GMT スナップショット
wavefield/paraview/	ParaView 用 VTU ファイル
wavefield/glvis/	GLVis 用バイナリファイル
material/	物性分布の出力
mesh.mesh	使用したメッシュ

3.3 結果の確認

3.3.1 サマリー

results/summary.txt にはシミュレーションの要約が記録される。以下に本例題の出力例を示す：

```

Discretization:
  Order: 4
  Elements: 1250
  DOFs: 20301

Time stepping:
  dt: 0.0001 s
  nt: 10000
  Simulation time: 1 s

Numerical stability:
  h_min: 20.00 m, h_max: 20.00 m
  CFL dt_max: 7.3535e-04 s (dt/dt_max = 0.14)
  PPW: 12.5 (f=30.0 Hz)

Performance:
  Setup time: 0.18 s
  Run time: 2.27 s
  Total time: 2.44 s
  DOFs/sec: 8.96e+07

```

特に CFL dt_max が設定した Δt より大きいことを確認する。 $\Delta t > \Delta t_{\max}$ の場合、計算は不安定となり発散する。PPW (Points Per Wavelength) は 1 波長あたりのグリッド点数であり、5 以上が一般的な目安である。

3.3.2 波動場スナップショット

付属の Python スクリプトで波動場を可視化できる：

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

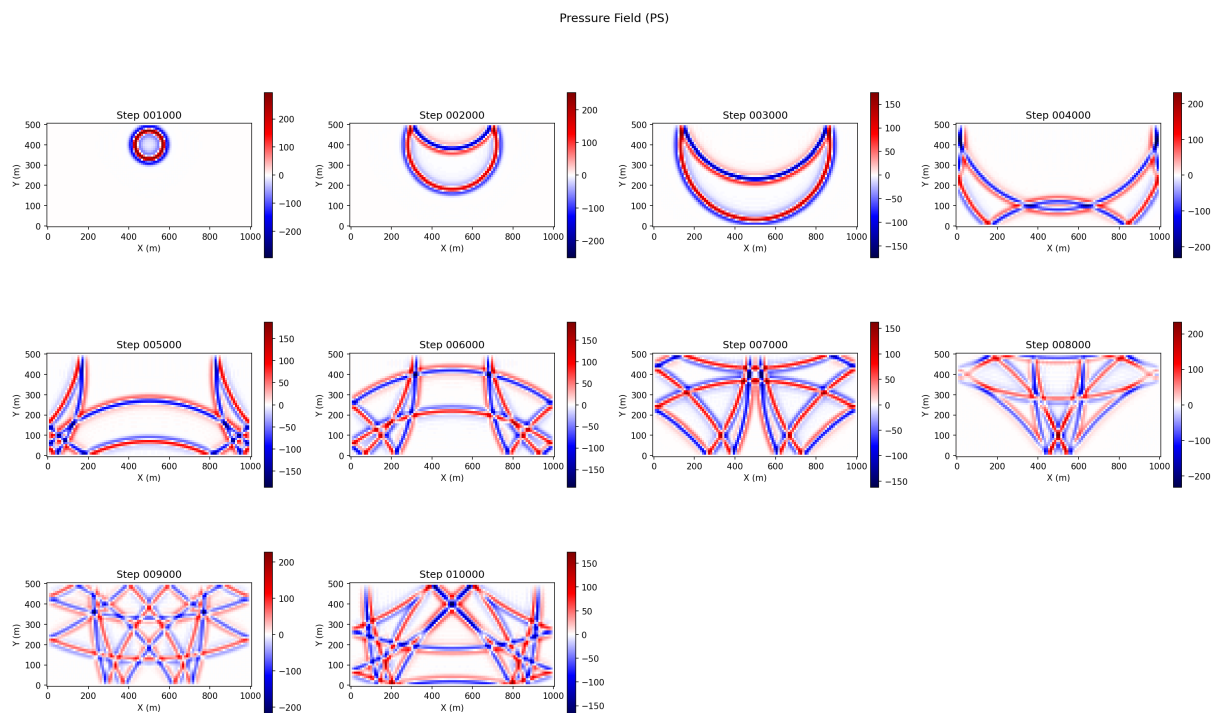


図 3.1 圧力場スナップショット（1000 ステップ間隔）。

本例題ではすべての辺に Dirichlet 境界条件を設定しているため、境界面で波が反射している様子が確認できる。実際の地震波シミュレーションでは吸収境界条件を使用して人工的な反射を抑制する。

3.3.3 受振器波形

受振器の波形ファイルは 2 列の ASCII テキスト形式である。1 列目が時刻（秒）、2 列目が振幅（圧力）である。gnuplot や Python など容易にプロットできる。

第 4 章

入力ファイルリファレンス

SEMSWS のシミュレーションは 1 つの YAML ファイル (`config.yaml`) ですべてのパラメータを指定する。本章では設定ファイルの全オプションを網羅的に記述する。基本的な使い方については第 3 章を参照されたい。

4.1 YAML 形式の基礎

設定ファイルは YAML (YAML Ain't Markup Language) 形式で記述する。以下に YAML の基本的な構文規則を示す。

- インデント（字下げ）で階層構造を表す。字下げには**スペースのみ**を使い（タブは不可）、同じ階層では字下げ幅をそろえる。
- `key: value` の形式でキーと値を書く（コロンの後にスペースが必要）。
- リストはブロック形式（各要素を `-` で始める）またはフロー形式 (`[a, b, c]`) で書く。マップも `{key: value}` のフロー形式が使える。
- `#`以降は行末までコメントとして無視される。
- 文字列は引用符なしでも書けるが、特殊文字を含む場合や、`yes/no/on/off` のように真偽値と解釈され得る語は `"..."` または `'...'` で囲む。

```
# スカラー 値
name: "My Simulation"
order: 4
dt: 0.0001

# ネストされた 構造
simulation:
  dimension: 2
```

```

steps: 10000
dt: 0.0001

# リスト
sides: [bottom, right, top, left]

# リスト（複数行形式）
list:
  - name: "R001"
    location: [400.0, 300.0]
  - name: "R002"
    location: [500.0, 300.0]

```

4.2 厳密キー検証 (Strict-Key Validation)

YAML ロードは設定ファイルの各ブロックを既知キー集合と照合する。どこかに未知のキー（タイプミス・リネーム済みの旧キー・古い設定ファイルからコピペされたサブブロックなど）があると、それを警告なく無視するのではなく、以下のような形式で実行時エラーとして中断する：

```

Error validating config: Unknown key `<bad_key>` in <section>
  (allowed: <そのブロックで有効なキーの一覧>)

```

4.3 設定ファイルの全体構造

設定ファイルの全体構造は以下の通りである：

```

name: "... "          # シミュレーション名（任意）

simulation:           # シミュレーション基本設定・出力
mesh:                # メッシュ
material:            # 物性
boundary:            # 境界条件
sources:             # 震源
receivers:           # 受振器
device:              # 実行デバイス
inversion:           # インバージョン設定（任意）
vis:                 # 可視化・波動場/物性/メッシュ出力（任意）

```

4.4 ルートレベルパラメータ

キー	型	必須	デフォルト	説明
name	string	任意	"Seismic wave simulation"	シミュレーション名。summary.txt に記録される

4.5 simulation セクション

4.5.1 基本パラメータ

キー	型	必須	値	説明
dimension	int	必須	2, 3	空間次元
order	int	必須	≥ 1	多項式次数 (GLL 点数 = $N + 1$)

GPU 実装では共有メモリ制約により order に上限がある。

4.5.2 時間積分

キー	型	必須	デフォルト	説明
dt	real	必須	—	時間刻み幅 (s)
steps	int	必須	—	時間ステップ数
cfl_factor	real	任意	0.3	CFL 安全係数 (正の実数。0.7 以下推奨)
t0	real	任意	0.0	シミュレーション開始時刻 (s)

Δt は CFL 条件 ($\Delta t < \Delta t_{\max}$) を満たす必要がある。cfl_factor はメッシュ品質評価に使用される。総シミュレーション時間は steps $\times$ Δt で決まる。

4.5.3 シミュレーションモード

キー	型	必須	デフォルト	説明
mode	string	任意	forward	forward, inversion, misfit_only

FWI 関連のパラメータ (`misfit_type`、チェックポイント設定、カーネル出力先など) は `simulation` ではなくトップレベルの `inversion` セクションに記述する。`inversion` は `mode` が `inversion` または `misfit_only` のときのみ参照される。許可キーは `misfit_type` (`l2_waveform / normalized_correlation`)、`kernel_dir`、`sensitivity_backend`、および `checkpoints` (サブキー `n / storage (memory / disk) / device (auto / host / device) / dir (storage: disk 時に必須)`) である。

`forward` が標準的なフォワードシミュレーションである。`inversion` と `misfit_only` は Full Waveform Inversion (FWI) 用である。`inversion` は checkpoint アルゴリズムと adjoint を用いた勾配計算を、`misfit_only` は波形の誤差計算を行う。現在は 2 次元音響波でのみ対応している。その他のキーも FWI 用であり、まだ十分にデバッグされていない。

4.5.4 出力設定

キー	型	必須	デフォルト	説明
<code>directory</code>	<code>string</code>	必須	—	出力ルートディレクトリ
<code>summary_file</code>	<code>string</code>	任意	<code>""</code>	サマリーファイル名 (空文字で無効)
<code>log_interval</code>	<code>int</code>	任意	99999	ログ出力間隔 (ステップ数)

全ての出力ファイルは `directory` で指定したディレクトリに出力される。`summary_file` には計算の要約が出力される (`summary.txt` などを指定しておくとい。空文字を指定すると何も出力されない)。`log_interval` は、計算の進行状況を何ステップごとに表示するかを指定する。進行状況を表示するたびに、並列計算では MPI 通信が、特に GPU 計算では Host-Device 間の通信が発生するため、間隔は適切な値に設定する。

4.6 vis セクション

波動場・物性値・メッシュの可視化出力はトップレベルの `vis` セクションで指定する (`simulation` の外側)。許可キーは `directory / wavefield / material / mesh`。

4.6.1 波動場出力 (vis.wavefield)

キー	型	必須	デフォルト	説明
enabled	bool	任意	false	波動場スナップショット出力を有効化
interval	int	条件	—	出力間隔 (ステップ数、enabled 時必須)
fields	string 配列	任意	["DISP"]	出力フィールド
formats	配列	条件	—	出力フォーマットのリスト (enabled 時必須)
components	int 配列	任意	[-1]	GMT 出力の成分指定 (全フォーマットに一括適用、GMT のみ使用)

formats の各要素は、次の 2 通りの書き方ができる。

- フォーマット名だけを書く (追加オプションが不要なとき)。例: glvis、paraview。
- type キーを持つマップとして書く (オプションを付けるとき)。フォーマット名を type に指定し、追加設定を同じ要素の中に並べる。

```
formats:
- glvis                      # 名前だけ (オプションなし)
- type: paraview             # マップ形式 (type + オプション)
  refinement: 2
- type: gmt                   # gmt は必ず マップ形式で書く
  resolution: [200, 200]
```

glvis と paraview はどちらの書き方でもよい。一方 gmt は名前だけでは指定できず、必ず type キーを持つマップ形式で書く (resolution、および 3D では cross_sections という必須オプションがあるため)。

fields に指定可能な値: DISP (変位)、VEL (速度)、ACC (加速度)、PS (圧力スカラー、acoustic wave simulation のときのみ)。

■出力フォーマット formats 配列の各要素に以下のオプションを指定する:

キー	対象	デフォルト	説明
type	全て	—	glvis, paraview, gmt
refinement	paraview	1	分割レベル
data_format	paraview	binary32	ascii, binary (64bit) , binary32 (32bit)
compression	paraview	-1	zlib 圧縮レベル (0-9, -1=デフォルト、おそらく zlib 付きで mfem のビルド必須、未テスト)
resolution	gmt	—	出力グリッド数 (解像度) [nx, ny]
cross_sections	gmt (3D)	—	断面位置

3次元 GMT 出力では `cross_sections` で断面位置を指定する：

```
cross_sections:
  yz: [2500.0]      # x=2500mでのyz断面
  xz: [2500.0]      # y=2500mでのxz断面
  xy: [1000.0]      # z=1000mでのxy断面
```

■サイドごとのオーバーライド (流体 – 固体連成用)。 `material.type: coupled` を使う場合、流体側と固体側は別々の `ParSubMesh` 上に定義されており、波動場出力ファイルもサイドごとに分けて書き出される。 `vis.wavefield:` ブロック内に `fluid:` や `solid:` のサブブロックを追加すると、 `enabled / interval / fields / formats / components` をそのサイドだけ上書きできる。指定しないキーは親の `wavefield:` 設定を継承する。片側で `enabled: false` とすると、そのサイドだけ出力を停止し、もう一方は通常通り書き出す。

```
wavefield:
  enabled: true
  interval: 500
  formats:
    - type: glvis
  fluid:
    fields: [PS]      # 流体側は圧力のみ
  solid:
    fields: [DISP, VEL] # 固体側は変位 + 速度
```

4.6.2 物性値出力 (vis.material)

キー	型	必須	デフォルト	説明
<code>enabled</code>	<code>bool</code>	任意	<code>false</code>	物性値出力を有効化
<code>fields</code>	<code>string</code> 配列	任意	<code>["vp"]</code>	出力フィールド
<code>formats</code>	配列	条件	—	フォーマット（波動場と同じ）

`fields` に指定可能な値：`vp`, `vs`, `rho`, `qkappa`, `qmu`。物性値出力はシミュレーション開始時に 1 回のみ書き出される。

`material.type`: `coupled` のシミュレーションでは、上記の波動場と同様に `vis.material` 配下にも `fluid`: / `solid`: サブブロックを配置して、サイドごとに `enabled` / `fields` / `formats` を上書きできる（固体側だけ `vs` を出す等）。

4.6.3 メッシュ可視化 (vis.mesh)

`vis.mesh` が持つキーは `enabled` のみで、`enabled: true` にするとメッシュ（分割の様子）の可視化出力が有効になる。

```
vis:
  mesh:
    enabled: true
```

4.6.4 simulation・vis の設定例

```
simulation:
  dimension: 3
  order: 4
  steps: 20000
  dt: 0.00005
  cfl_factor: 0.3
  directory: "./results"
  summary_file: "summary.txt"
  log_interval: 500

vis:
  wavefield:
    enabled: true
```

```

interval: 1000
fields: ["DISP"]
components: [-1, 0, 1, 2]
formats:
  - type: "paraview"
    refinement: 2
    data_format: "binary32"
  - type: "gmt"
    resolution: [200, 200]
    cross_sections:
      xy: [0.0]
      xz: [2500.0]

material:
  enabled: true
  fields: ["vp", "vs", "rho"]
  formats:
    - type: "paraview"
      refinement: 2

```

4.7 mesh セクション

4.7.1 共通パラメータ

キー	型	必須	デフォルト	説明
type	string	必須	—	internal, external, prepart_mfem
max_freq	real	必須	—	メッシュ品質評価用の最大周波数 (Hz)
ppw	real	必須	—	Points Per Wavelength

4.7.2 内部メッシュ (type: internal)

SEMSWS 内部で構造格子メッシュを自動生成する。

キー	型	必須	デフォルト	説明
<code>origin</code>	real 配列	必須	—	原点座標 [x,y] or [x,y,z] (m)
<code>size</code>	real 配列	必須	—	ドメインサイズ [Lx,Ly] or [Lx,Ly,Lz] (m)
<code>elements</code>	int 配列	必須	—	要素数 [Nx,Ny] or [Nx,Ny,Nz]
<code>partition_method</code>	string	任意	<code>metis</code>	<code>metis</code> or <code>cartesian</code>
<code>partition_grid</code>	int 配列	条件	—	<code>cartesian</code> 時の分割数 [nx,ny,nz]

`partition_method`: `cartesian` は METIS 不要の構造的分割である。`partition_grid` では三次元で [4, 4, 2] と指定すると x, y, z 軸方向にそれぞれ 4, 4, 2 と分割される。このとき実行コマンドの並列数 (`-np 4` など) は、分割数の積である 32 と一致していなければならない。

4.7.3 外部メッシュ (type: external)

キー	型	必須	デフォルト	説明
<code>file</code>	string	必須	—	メッシュファイルパス (拡張子から MFEM が形式を自動判定)
<code>partition_method</code>	string	任意	<code>metis</code>	MPI 分割方法 (<code>metis</code> or <code>cartesian</code>)

```
mesh:
  type: external
  file: "model.exo"           # GMSH(.msh) / Exodus(.exo) など。拡張子で自動判別
  partition_method: metis     # metis または cartesian
```

ExodusII 形式 (CUBIT) を使用する場合、MFEM のビルド時に `MFEM_USE_NETCDF=ON` が必要である。

4.7.4 事前分割メッシュ (type: prepart_mfem)

`partition_mesh` ツールで事前分割した MFEM フォーマットのメッシュを読み込む。各 MPI ランクが自身の分割ファイルのみを読む。大規模メッシュや何度も同じメッシュで計算する際に有用。

キー	型	必須	デフォルト	説明
<code>prepart_mfem.directory</code>	string	必須	—	分割ファイルのディレクトリ
<code>prepart_mfem.nparts</code>	int	必須	—	分割数 (MPI プロセス数と一致)

ファイル命名規則は `mesh.mesh.000000`, `mesh.mesh.000001`, ... である。

4.7.5 mesh セクションの設定例

内部メッシュ:

```
mesh:
  type: internal
  origin: [0.0, 0.0, 0.0]
  size: [5000.0, 5000.0, 3000.0]
  elements: [50, 50, 30]
  max_freq: 20.0
  ppw: 5.0
  partition_method: cartesian
  partition_grid: [4, 4, 2]
```

外部メッシュ:

```
mesh:
  type: external
  file: "./meshes/model.exo"
  max_freq: 15.0
  ppw: 5.0
```

事前分割メッシュ:

```
mesh:
  type: prepart_mfem
  prepart_mfem:
    directory: "./meshes/partitioned"
    nparts: 32
  max_freq: 15.0
  ppw: 5.0
```

4.8 material セクション

4.8.1 物性タイプとフォーマット

type の値	説明
isotropic_acoustic	等方性音響媒質 (vp, rho)
isotropic_elastic	等方性弾性媒質 (vp, vs, rho)
coupled	流体 – 固体連成媒質。流体側は isotropic_acoustic、固体側は isotropic_elastic。fluid: と solid: のサブブロックで両者を独立に宣言する。詳細は §4.8.8。

format の値	説明
constant	YAML 内で均質物性を直接指定
grid	構造格子の ASCII ファイル
by_attribute	メッシュ属性ごとのテキストファイル
by_attribute_mixed	属性ごとに constant/grid を混合指定する YAML ファイル
adios2	ADIOS2 BP ファイル (FWI 用)

type: coupled の場合は最上位で format を指定しない。ネストされる fluid: / solid: サブブロックがそれぞれ独自の format を持つ。

4.8.2 format: constant

YAML 内で物性値を直接指定する。

キー	型	elastic	acoustic
vp	real	必須	必須
vs	real	必須	—
rho	real	必須	必須

単位はそれぞれ m/s, m/s, kg/m³。

```
material:
  type: isotropic_elastic
```

```
format: constant
vp: 6000.0
vs: 3500.0
rho: 2700.0
```

4.8.3 format: grid

構造格子上の不均質物性を ASCII ファイルから読み込む。file にファイルパスを指定する。

ファイルフォーマット：

```
Nx Ny [Nz]          # 格子点数
dx dy [dz]          # 格子間隔 (m)
x0 y0 [z0]          # 原点 (m)
vp vs rho [Qkappa Qmu] # 各格子点の物性値
vp vs rho [Qkappa Qmu] # (Nx*Ny[*Nz]行)
...
```

弾性の場合は 5 列 (vp, vs, rho, Qkappa, Qmu)、音響の場合は 3 列 (vp, rho, Qkappa)。Q 値の列は減衰が有効な場合のみ必要。

データは、x 方向の添字が最も速く変化する順に格納する（次に y、3 次元ではその次に z）。インデックスと座標の対応は以下の通り：

■2 次元 i 行目のデータは格子点 (i_x, i_y) に対応する。

$$i = i_y \times N_x + i_x$$

$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy$$

すなわち、 x 方向に N_x 点走査してから y 方向に 1 つ進む。

■3 次元 i 行目のデータは格子点 (i_x, i_y, i_z) に対応する。

$$i = i_z \times N_x \times N_y + i_y \times N_x + i_x$$

$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy, \quad z = z_0 + i_z \times dz$$

すなわち、 $x \rightarrow y \rightarrow z$ の順で走査する。グリッド外の点は最近傍の境界値にクランプされる。

原点 (x_0, y_0) または (x_0, y_0, z_0) はグリッドの最小座標点であり、通常はメッシュの origin と一致させる。semsws の座標系では y 軸 (2D) / z 軸 (3D) が上向き正であるため、原点はドメインの最深部に位置する。したがってファイルの先頭行は最も深い場所の物性値から始まる。

4.8.4 format: by_attribute

メッシュの要素属性 (attribute) ごとに物性を定義するテキストファイル。外部メッシュ (CUBIT 等) で属性を付与した場合に使用する。

#	attr	vp	vs	rho	[Qkappa	Qmu]
1		6000.0	3500.0	2700.0	100.0	100.0
2		3000.0	2000.0	2200.0	50.0	50.0

4.8.5 format: by_attribute_mixed

属性ごとに constant (均質)、grid (格子ファイル)、または adios2 (属性ごとの BP ファイル) を混合指定する外部 YAML ファイル。mode: adios2 の各属性は vp_file / rho_file (弾性では vs_file、減衰時は qkappa_file / qmu_file) を持つ。

```
attributes:
  1:
    mode: constant
    vp: 5500.0
    vs: 3175.0
    rho: 2800.0
  2:
    mode: grid
    file: material_layer2.dat
```

4.8.6 format: adios2

FWI の反復で生成された GLL 点上の物性データを ADIOS2 BP 形式で読み込む。

キー	型	必須	説明
vp_file	string	必須	Vp の BP ファイルパス
vs_file	string	条件付	Vs の BP ファイルパス (type: isotropic_elastic で必須)
rho_file	string	必須	ρ の BP ファイルパス
attenuation.files.qkappa	string	任意	Q_κ の BP ファイルパス (粘性音響・粘弾性で必要)
attenuation.files.qmu	string	任意	Q_μ の BP ファイルパス (粘弾性で必要)

`type: coupled` の `fluid:` / `solid:` サブブロックでは `adios2` フォーマットは現在サポートされていない。

4.8.7 減衰 (attenuation)

一般化 Zener モデル (Standard Linear Solid: SLS) による減衰を設定する。

キー	型	必須	デフォルト	説明
<code>attenuation.enabled</code>	bool	任意	false	減衰の有効化
<code>attenuation.f0</code>	real	条件	1.0	参照周波数 (Hz)。enabled=true で必須
<code>attenuation.n_units</code>	int	条件	3	SLS 緩和機構の数。enabled=true で必須
<code>attenuation.qkappa</code>	real	条件	—	体積弾性率の Q 値 (format: constant 時に必須)
<code>attenuation.qmu</code>	real	条件	—	せん断弾性率の Q 値 (format: constant かつ弾性で必須)
<code>attenuation.files.qkappa</code>	string	任意	—	Q_κ の BP ファイル (format: adios2 で Q をファイル指定する場合)
<code>attenuation.files.qmu</code>	string	任意	—	Q_μ の BP ファイル (弾性で同上)

Q 値のフィッティング帯域は $[0.1f_0, 10f_0]$ である。減衰を使う場合、format によらず `enabled · f0 · n_units` は必須である。一方 format: grid や by_attribute では Q 値がファイルの列から取られるため、スカラーの `attenuation.qkappa/attenuation.qmu` は指定しない (指定するとエラーになる)。明示的に別の BP ファイルから Q を読み込みたい場合のみ、`attenuation.files.qkappa/attenuation.files.qmu` を指定する。

4.8.8 type: coupled (流体 – 固体連成)

流体 (音響) 領域と固体 (弾性) 領域が界面を共有するシミュレーションでは、`material.type: coupled` を指定する。この場合、最上位の `material:` ブロックは `fluid:` と `solid:` の二つのサブブロックを持ち、それぞれが独自の `type · format · 物性パラメータ` を宣言する (両者は独立であり、例えば流体を constant、固体を grid にすることもできる)。

キー	型	必須/オプション	説明
attribute	int	必須	このサブ領域を識別する親メッシュ要素の属性値。 fluid と solid で異なる値にすること。
type	enum	必須	fluid では isotropic_acoustic、solid では isotropic_elastic を指定。内部で再度 coupled を指定することはできない。
format	enum	必須	通常材料と同じ値: constant、grid、by_attribute、by_attribute_mixed。
vp, vs, rho	real	条件付	format: constant の場合に必須（弾性側は vs も必須）。
file	path	条件付	format: grid / by_attribute / by_attribute_mixed の場合に必須。
attenuation	block	オプション	attenuation: サブブロックを持つ場合、非連成材料と同じキー構成 (§4.8.7) を用いる。fluid: 側で有効化すると粘性音響、solid: 側で有効化すると粘弾性となる。

流体 – 固体界面の境界属性は実行時に FluidSolidInterface::Setup が自動検出する（二つの ParSubMesh のいずれか一方にのみ存在する境界属性のうち、親メッシュが元々持つ外側境界には含まれないものが界面として抽出される）。YAML 側で明示的に指定する必要はない。

■連成サブブロックで使用可能な format. fluid: / solid: サブブロックで指定できる format は constant / grid / by_attribute / by_attribute_mixed の 4 つに限られる。adios2 は連成サブブロックでは現在サポートされていない（スキーマ上は受理されるが実行時に abort する）。

```
# 2次元の流体-固体連成。両側を均質物性とし、
# 減衰 ( $Q_{\kappa} = Q_{\mu} = 40$ ) を各領域で有効化する例。
material:
  type: coupled

  fluid:
    attribute: 1
    type: isotropic_acoustic
    format: constant
    vp: 1500.0
    rho: 1000.0
    attenuation:
```

```
    enabled: true
    f0: 5
    n_units: 3
    qkappa: 40

solid:
  attribute: 2
  type: isotropic_elastic
  format: constant
  vp: 3000.0
  vs: 1732.0
  rho: 2500.0
  attenuation:
    enabled: true
    f0: 5
    n_units: 3
    qkappa: 40
    qmu: 40
```

■メッシュ側の要件。 親メッシュは流体要素すべてに `fluid.attribute` の値を、固体要素すべてに `solid.attribute` の値をタグ付けしておく必要がある（Cubit / GMSH では物理グループで判別する）。加えて、流体要素と固体要素は界面で位相的に接続されている必要がある。

4.8.9 モデルエクスポート

モデルのエクスポートは `material:` 内のキーではなく、専用ツール `semsws_export_model` で行う。実行時に `-config <config.yaml>` で設定ファイルを、`-out <出力ディレクトリ>` で出力先を指定する。出力は `vp.bp` / `vs.bp` / `rho.bp`（減衰有効時は `qkappa.bp` / `qmu.bp`）の ADIOS2 BP ファイル群である。

4.8.10 material セクションの設定例

均質弾性体（減衰あり）:

```
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
```



```
attenuation:
  enabled: true
  f0: 2.0
  n_units: 3
  qkappa: 100.0
  qmu: 50.0
```

属性ごとの定義（外部メッシュ使用時）：

```
material:
  type: isotropic_elastic
  format: by_attribute
  file: "materials.txt"
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
```

不均質音響媒質（グリッドファイル）：

```
material:
  type: isotropic_acoustic
  format: grid
  file: "velocity_model.dat"
```

4.9 boundary セクション

4.9.1 吸収境界条件（absorbing）

Cerjan スポンジ層による吸収境界条件を設定する。absorbing ブロック自体は省略可能で、省略時は吸収境界なし（sides: [] と等価）となる。

キー	型	必須	デフォルト	説明
sides	string 配列	任意	[]	適用面（空配列 [] で無効化）
thickness	real	条件付き	—	スポンジ層の厚さ（m）。sides が非空のとき必須
alpha	real	条件付き	—	減衰係数（非負）。sides が非空のとき必須

sides に指定可能な面名：

次元	指定可能な面
2D	bottom, right, top, left
3D	bottom, front, right, back, left, top

■面名と境界属性 ID の対応。 SEMSWs では各面名が固定された 1 始まりの MFEM 境界属性 ID に変換される。この ID は `sides` でも `dirichlet` リストでも同じ表を用いる。*Cubit*・*GMSH* などの外部メッシュジェネレータでメッシュを作成する場合は、境界面にこの ID をそのまま付与する必要がある。ID がズレると YAML 側の面名が正しい物理境界に対応しない。

表 4.1 面名と SEMSWs が使う MFEM 境界属性 ID の対応。 `Mesh::MakeCartesian2D`・`MakeCartesian3D` が生成する ID であり、外部メッシュを作る際の面タグ付けでもこの値に従う必要がある。

面名	2D 属性 ID	3D 属性 ID
bottom	1	1 (z 最小)
right	2	3 (x 最大)
top	3	6 (z 最大)
left	4	5 (x 最小)
front	—	2 (y 最小)
back	—	4 (y 最大)

`dirichlet` リストは面名と整数属性 ID の両方を混在して受け付ける。面名は上表の整数 ID の別名（エイリアス）にすぎない。

4.9.2 Dirichlet 境界条件 (dirichlet)

キー	型	必須	デフォルト	説明
<code>dirichlet</code>	配列	任意	<code>[]</code>	面名または整数の境界属性 ID (例 <code>dirichlet: [top, left]</code>)。旧形式の <code>attributes</code> : ラップは廃止

面名 (`top`, `left` 等) と整数属性 ID を混在させることができる。ID の付与には外部メッシュソフト (*CUBIT*/*GMSH* など) が必須。内部メッシュではメッシュ領域の境界のみ `top`, `bottom`.... などで指定可能。SEM では未指定の境界は自然境界条件となるので、音響波動シミュレーションの際は `dirichlet` が必須となる。

4.9.3 boundary セクションの設定例

```
# 海域シミュレーション：海面を Dirichlet、その他を吸収境界
boundary:
  absorbing:
    sides: [bottom, left, right]
    thickness: 1200.0
    alpha: 5.0
  dirichlet: [top]
```

4.10 sources セクション

4.10.1 震源モードと入力形式

キー	型	必須	デフォルト	説明
list	配列	条件	—	インライン震源定義 (file とはどちらか一方)
file	string	条件	—	SEMSWS HDF5 v2.0 ファイルパス (list とはどちらか一方)
shot_id	int	任意	0	file 指定時の対象 shot ID (≥ 0)

`sources.list` に列挙した全震源は同一シミュレーション内で同時（重ね合わせ）に発火する。震源ごとに個別発火するモードを選ぶ設定キーは存在しない。

■インライン定義 (list:) . インラインの `sources.list` に震源を直接記述する。

■HDF5 ファイル (file:) . `file` を指定すると、SEMSWS HDF5 v2.0 schema から `shot_id` で指定した 1 shot 分の震源情報を読み込む。list と file はどちらか一方だけを指定し、両方指定するとエラーになる。詳細スキーマ・制約・使い方は §4.14 を参照。

4.10.2 震源の共通パラメータ (list)

`sources.list` 配列の各要素に以下を指定する：

キー	型	必須	デフォルト	説明
id	int	任意	i+1	震源 ID（未指定時は配列内の位置に基づき自動採番）
name	string	任意	—	震源名（現在のバージョンでは使用されない）
type	string	必須	—	force, pressure, moment_tensor
location	real 配列	必須	—	位置 [x,y] or [x,y,z] (m)

4.10.3 震源タイプ: force

弾性波シミュレーション用の single force。

キー	型	必須	説明
direction	real 配列	必須	力の方向ベクトル [x,y] or [x,y,z]

```
- id: 1
  type: force
  location: [500.0, 300.0, 0.0]
  direction: [0.0, 0.0, 1.0]    # 鉛直方向
  wavelet:
    type: ricker
    frequency: 10.0
    amplitude: 1.0e10
```

4.10.4 震源タイプ: pressure

音響波シミュレーション用の圧力点震源。追加パラメータなし。

4.10.5 震源タイプ: moment_tensor

モーメントテンソル震源。地震のメカニズム解を直接指定する。

3次元の場合（6成分）: Mxx, Myy, Mzz, Mxy, Mxz, Myz。

2次元の場合（3成分）: Mxx, Myy, Mxy。

```
- id: 1
  type: moment_tensor
  location: [5000.0, 5000.0, 3000.0]
```

```

moment_tensor:
  Mxx: 1.0e18
  Myy: 0.0
  Mzz: 0.0
  Mxy: 0.0
  Mxz: 0.0
  Myz: 0.0
wavelet:
  type: ricker
  frequency: 5.0
  amplitude: 1.0

```

4.10.6 ウェーブレット (wavelet)

キー	型	必須	デフォルト	説明
type	string	任意	ricker	ricker, gaussian, external
frequency	real	条件	—	主周波数 (Hz)。ricker/gaussian で必須
amplitude	real	条件	—	振幅スケーリング。ricker/gaussian で必須 (external では無視され 1.0 固定)
delay	real	任意	0.0	ピーク到達時刻の遅延 (s)
file	string	条件	—	外部 STF ファイルパス (external 時に必須)

Ricker ウェーブレットでは $\text{delay} \geq 1.2/\text{frequency}$ を推奨する。

外部 STF ファイルは 2 列の ASCII テキスト (時刻, 振幅) である (時刻列は無視されるため、1 行目の振幅がシミュレーション開始時刻に対応するよう揃えておく必要がある) :

```

# time (s)      amplitude
0.0000          0.000000e+00
0.0001          1.234568e-05
...

```

4.10.7 観測データ — インバージョンモード

`simulation.mode` が `inversion` または `misfit_only` の場合、観測データは `sources.file` で指定した SEMSWS HDF5 v2.0 ファイル (`sources.shot_id` で選択した 1 shot 分) から読み込まれる。このモードでは `sources.file` が必須である。専用の `observed:` キーは存在しない。

SU/SEG-Yなどを観測データとして使う場合は事前に SEMSWS HDF5 v2.0 形式へ変換しておく。

4.10.8 sources セクションの設定例

音響波（複数震源、同時発火）：

```
sources:
  list:
    - id: 1
      type: pressure
      location: [300.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
    - id: 2
      type: pressure
      location: [700.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

弾性波（モーメントテンソル、外部 STF）：

```
sources:
  list:
    - id: 1
      type: moment_tensor
      location: [5000.0, 5000.0, 3000.0]
      moment_tensor:
        Mxx: 1.0e18
        Myy: -1.0e18
        Mzz: 0.0
        Mxy: 0.0
        Mxz: 0.0
        Myz: 0.0
      wavelet:
        type: external
```

```
frequency: 5.0
file: "stf_earthquake.txt"
```

外部 HDF5 ファイルを使う場合は `file` で HDF5 ファイルを指定する (`list` とはどちらか一方):

```
sources:
  file: "./sources.h5"
  shot_id: 0
```

4.11 receivers セクション

4.11.1 受振器の指定方法

受振器は次の 2 通りで指定でき、記録タイプ (`type`) と出力設定 (`output`) はどちらでも共通である。

■インライン (`list / line`) . YAML に直接記述する。`list` で受振器を個別に列挙するか、`line` で等間隔の受振器列を自動生成する。

■HDF5 ファイル (`sources.file`) . `sources.file` を指定すると、そのファイル内の受振器位置・タイプが使われる (主に FWI)。この場合 `receivers` セクションは省略できる。詳細スキーマ・使い方は §4.14 を参照。

4.11.2 記録タイプ (`type`) と出力設定 (`output`)

キー	型	必須	デフォルト	説明
<code>type</code>	string/配列	必須	—	既定の記録タイプ (受振器ごとに上書き可、後述)
<code>output.formats</code>	配列	必須	—	出力形式のリスト (平文文字列 <code>ascii/hdf5/su</code>)
<code>output.filename</code>	string	任意	<code>shot</code>	<code>hdf5/su</code> のファイル名 (<code>ascii</code> では無視)
<code>output.buffer_steps</code>	int	任意	—	出力をフラッシュする前にバッファするステップ数 (≥ 0)

`type` に指定可能な値:

値	説明	対応モデル
DISP	変位	elastic
VEL	速度	elastic
ACC	加速度	elastic
PS	圧力	acoustic
GRAD	ひずみ（勾配、順解析のみ）	elastic

■出力形式。 `output.formats` は文字列のリストで、各要素に `ascii`（テキスト、受振器ごと 1 ファイル）、`hdf5`（単一 HDF5 ファイル）、`su`（Seismic Unix 形式）のいずれかを直接書く。複数形式を同時に要求できる：

```
output:
  formats:
    - ascii
    - su
  filename: "seismograms"      # hdf5 / su のファイル名（任意、既定 shot）
```

出力された `su` ファイルを Seismic Unix で可視化する場合は、`XDRFLAG` を `OFF` にしてビルドされた Seismic Unix が必要。

4.11.3 受振器リスト（list）

`list` には各受振器を列挙する。各要素のキーは次のとおり。

キー	型	必須	デフォルト	説明
<code>name</code>	string	必須	—	受振器名
<code>location</code>	real 配列	必須	—	位置 <code>[x,y]</code> or <code>[x,y,z]</code> (m)
<code>weight</code>	real	任意	1.0	スカラ重み（FWI 観測データ組立てで使用）

4.11.4 受振器ライン（line）

等間隔に配置された受振器列を自動生成する。

キー	型	必須	デフォルト	説明
<code>line.start</code>	real 配列	必須	—	開始位置 (m)
<code>line.end</code>	real 配列	必須	—	終了位置 (m)
<code>line.count</code>	int	必須	—	受振器数

4.11.5 receivers セクションの設定例

リスト形式（個別指定）：

```
receivers:
  type: [DISP, VEL]
  output:
    formats:
      - ascii
      - su
    filename: "seismograms"
  list:
    - name: "R01"
      location: [1000.0, 2500.0, 0.0]
    - name: "R02"
      location: [2000.0, 2500.0, 0.0]
    - name: "R03"
      location: [3000.0, 2500.0, 0.0]
```

ライン形式（等間隔自動生成）：

```
receivers:
  type: [PS]
  output:
    formats:
      - ascii
  line:
    start: [100.0, 300.0]
    end: [900.0, 300.0]
    count: 17
```

流体－固体連成での受振器配置例。全タイプの union ([PS, DISP, VEL, ACC]) を指定しておく
と、各受振器はその位置する側で物理的に有効なタイプだけが自動的に記録される（R001 は流体側
なので PS のみ、R002/R003 は固体側なので DISP/VEL/ACC のみ。側に合わないタイプは自動的に
除外され、ログに出力される）：

```

receivers:
  output:
    formats:
      - ascii
  type: [PS, DISP, VEL, ACC]
  list:
    - name: "R001"
      location: [800.0, 200.0]
    - name: "R002"
      location: [800.0, 440.0]
    - name: "R003"
      location: [800.0, 680.0]

```

4.12 device セクション

キー	型	必須	デフォルト	説明
type	string	必須	—	cpu, omp, cuda, hip, raja-cpu, raja-omp, raja-cuda, raja-hip

type はビルド構成と一致させる必要がある。GPU ビルドで cpu を指定すると動作するが非常に低速であるため、CPU 実行には CPU ビルドを使用すべきである。

GPU 上での波形記録バッファのステップ数は `receivers.output.buffer_steps` で指定する (device セクションには指定しない)。

4.12.1 device セクションの設定例

```

device:
  type: cuda

```

4.13 inversion セクション

`simulation.mode` が `inversion` または `misfit_only` のときに任意で配置できるトップレベルセクション。感度カーネルの算出経路を切り替える。

キーはすべて `inversion:` 配下のもの。デフォルトは説明欄に併記する。

キー	型	必須	説明
sensitivity_backend	string	任意	感度カーネルのバックエンド (既定 hand)。hand (手書きの随伴経路、production) または ad (forward-mode 自動微分による検証用経路)
misfit_type	string	任意	ミスフィット関数 (既定 l2_waveform)。l2_waveform または normalized_correlation
kernel_dir	string	任意	感度カーネル出力ディレクトリ (既定 <simulation.directory>/kernels)
checkpoints.n	int	任意	チェックポイント数 (既定 10、 ≥ 1)
checkpoints.storage	string	任意	チェックポイント保存先 (既定 memory)。memory または disk (disk 時は dir 必須)
checkpoints.device	string	任意	チェックポイントのデバイス (既定 auto)。auto / host / device
checkpoints.dir	string	任意	storage: disk 時の保存ディレクトリ (既定 <simulation.directory>/checkpoints)

ad は手書き実装 (hand) と数値的に一致するかを検証するための代替経路で、計算コストは大きい。通常は既定の hand のままで良い。

```
inversion:
    sensitivity_backend: hand    # or "ad" for AD verification
```

4.14 HDF5 入出力スキーマと使い方

SEMSWS は震源・受振器・観測データの入力、および受振器波形の出力に YAML とは別に HDF5 v2.0 schema 経由のパスをサポートする。多 shot 構成の FWI ワークフロー (semsws-driver が外側でループ) や、観測データを読み込むインバージョンモードでは、HDF5 が事実上必須の入力形式となる。

4.14.1 HDF5 v2.0 スキーマ全体像 (共通)

すべての入力 / 出力ファイルはルート属性で format_version="2.0"を持ち、/shots/<NNNN>/ (<NNNN>は 4 桁ゼロ詰め shot ID) 配下に 1 shot 分の震源と受振器をまとめる。

```
/         attrs: format_version="2.0",
          dt, t0, n_samples, space_dim,
```

```

                                coord_system="cartesian", units="
                                SI"
/shots/<NNNN>/                attrs: shot_id (i32)
                                /sources/<key>/          attrs: id (i32), type (str),
                                                                position (f64[space_dim]),
                                                                direction (f64[space_dim]; force用
                                                                ),
                                                                label (str; optional)
                                /moment_tensor            (6,) or (3,)          f64    # type=
                                moment_tensor用
                                                                attrs: component_order (var-length str
                                                                array; 必須)
                                /stf                      (n_samples,)          f64    # 時系列
                                STF (F5: f64必須、shape=(n_samples,))
                                /receivers/<key>/          attrs: position (f64[space_dim])
                                                                types (var-length str array;
                                                                optional)
                                                                label (str; optional)
                                /PS                        (nt,)                f32    # type=PS
                                の波形
                                /VEL                      (space_dim, nt)         f32    # type=
                                VEL の波形
                                /DISP                    (space_dim, nt)         f32    # type=
                                DISP の波形
                                /ACC                      (space_dim, nt)         f32    # type=
                                ACC の波形
                                /weight_<ch>            same shape as <ch>       f32    # オブ
                                ション、observed側の重み (<ch>=PS/VEL/DISP/ACC)

```

■shot と source の関係. /shots/<NNNN>/は1回の発震イベント (= 1回のシミュレーション) を表し、その配下の/sources/に属する震源は**すべて同時発火される**。受振器には全 source を重ね合わせた合成波形が1セットだけ記録される。逐次発火に切り替えるキー (sources.mode のようなもの) は存在しない。source 別に分けて記録したい場合は shot_id を変えて shot を分ける (semsws-driver の shot 間ループが担当)。

■グループ名 <key> の規約.

- 震源: S<NNNN> (4桁ゼロ詰め)。@id 属性と一致しなければならない (@id=42 なら S0042)。reader はこれを strict に validate する。
- 受振器: 任意の文字列でよい (命名規約なし)。各受振器の**名前**は、@label 属性があればそ

の値、なければグループ名を採用し、(1) ASCII 出力のファイル名 (<名前>_<shot_id>.p 等)、(2) HDF5 出力時の/receivers/<名前>/グループ名、(3) ログ・診断メッセージ、に使われる。

■属性・データセットの説明. 以下で型の f64[d] は d=space_dim (空間次元) 長のベクトルを表す。

表 4.2 ルート属性

属性	型	必須	説明
format_version	str	必須	スキーマ版 ("2.0")
dt	f64	必須	時間刻み幅 (s)
t0	f64	必須	開始時刻 (s)
n_samples	i64	必須	時系列のサンプル数
space_dim	i32	必須	空間次元 (2 または 3)

表 4.3 shot・震源 (source) の属性とデータセット

属性 / データセット	型	必須	説明
@shot_id	i32	必須	/shots/<NNNN>/グループの属性。shot (発震イベント) の識別子。出力ファイル名 suffix と /shots/<NNNN>/キーに反映
@id	i32	必須	震源 ID (正の整数、shot 内で一意)。グループ名 S<NNNN>と一致必須
@type	str	必須	震 源 タ イ プ : force / pressure / moment_tensor
@position	f64[d]	必須	震源位置の座標
@direction	f64[d]	条件	力の方向 (type=force のとき必須)
@label	str	任意	表示名 (任意のメタデータ)
/moment_tensor	f64 (6,)/(3,)	条件	モ ー メ ン ト テ ン ソ ル 成 分 (type=moment_tensor のとき必須)。各値の成分名を示す@component_order (必須) を伴う
/stf	f64 (n_samples,)	必須	震源時間関数 (source time function) の時系列

表 4.4 受振器 (receiver) の属性とデータセット

属性 / データセット	型	必須	説明
@position	f64[d]	必須	受振器位置の座標
@types	str 配列	任意	この受振器が記録するチャンネル (PS/VEL/DISP/ACC)。省略時は親の receivers.type を継承し、両方無いとエラー
@label	str	任意	表示名 (上記「グループ名の規約」参照)
/PS	f32 (nt,)	—	圧力スカラー波形 (types に PS を含むとき)
/VEL, /DISP, /ACC	f32 (d, nt)	—	速度・変位・加速度のベクトル波形
/weight_<ch>	f32	任意	observed 側のチャンネル別重み (<ch>=PS/VEL/...)

■モーメントテンソルの指定 (component_order) . type=moment_tensor の震源では、/moment_tensor に独立成分の値を 1 次元配列として格納し、同じ長さの文字列配列

@component_order で各値がどの成分かを宣言する。有効な成分名は、3D で Mxx, Myy, Mzz, Mxy, Mxz, Myz (6 個)、2D で Mxx, Myy, Mxy (3 個)。**並び順は自由**で、@component_order がデータ配列と位置ごとに対応していればよい。全成分をちょうど 1 回ずつ含む必要があり、重複・無効名・個数不一致はエラーとなる。

4.14.2 入力経路: HDF5 → SEMSWS

震源を HDF5 から渡す (sources.file)

```
sources:
  file: inputs/shots.h5          # ← file を指定すると HDF5 入力経路 (format キーは無い)
  shot_id: 0                     # /shots/0000/sources/ を読む
```

sources.shot_id で読み込み対象の shot 番号を指定する。震源の type / position / direction / moment_tensor と STF 時系列がすべて HDF5 から取得されるため、YAML 側の sources.list は記述しない (sources.file と sources.list はどちらか一方だけを指定し、両方指定するとエラーになる)。

重要: sources.file 経路では STF は時系列データとして必ず /shots/<NNNN>/sources/<key>/stf データセットに格納されていなければならない。wavelet.type: ricker のような YAML による解析ウェブレットの指定は sources.file と併用できない。Ricker などを使いたい場合も、対応する離散時系列を事前に計算して HDF5 に書き込んでおく。

受振器位置を HDF5 から渡す (sources.file から自動取得)

```
# 受振器の位置は sources.file の HDF5 ファイルから自動取得される
# (receivers 側に file/format/shot_id キーは無い)。
sources:
  file: inputs/shots.h5
  shot_id: 0                     # /shots/0000/receivers/ も同じファイル・同じ shot_id から読む
receivers:
  output:
    formats: [ascii]
    filename: "seis"
```

sources.file を指定し、receivers 側に list:/line: がない場合、受振器位置は同じ HDF5 ファイル (/shots/<NNNN>/receivers/) から取得される。この経路で読まれるのは各受振器の @position / @types / @label のみで、/PS・/VEL などの波形データセットは不要である (forward 実行では受振器の幾何情報だけで十分なため)。これらの波形データセットは、観測データとしてイ

ンバージョンで読み込む場合にのみ必要となる。

観測データ（インバージョン専用）

`simulation.mode: inversion` または `misfit_only` では、各震源に対する観測波形を以下のよう指定する:

```
# inversion / misfit_only モードでは観測データも sources.file の
# HDF5ファイル (/shots/<shot_id>/receivers/<key>/ 下のPS/VEL/DISP/ACC
# データセット) から読み込まれる。専用の observed: ブロックは無い。
sources:
  file: obs/shot_0000.h5          # 震源・受振器・観測波形を含むv2.0フ
    イル (必須)
  shot_id: 0
```

観測ファイルは §4.14.1 のスキーマに従って `/shots/<NNNN>/receivers/<key>/` 下のデータセット (PS, VEL, DISP, ACC) を参照する。受振器の `@position` 属性とシミュレーション側の受振器位置が一致しないと、`misfit` (残差) が正しく計算されない。

入力 HDF5 構造の例

ユースケース別に典型的な入力 HDF5 の中身を3例示す。いずれも `root` 属性(`format_version=2.0`, `dt`, `t0`, `n_samples`, `space_dim`, ...) は省略表記している。

■例 1: 単震源・単 shot (典型的な海底地震計シミュレーション) .

```
inputs/shot.h5
└─ /shots/0000/                                attrs: shot_id=0
    └─ /sources/
        └─ /S0001/                                attrs: id=1, type="pressure",
            │                                     position=[5000, 4000]
            └─ /stf                                (n_samples,) ← 事前計算した時系
                列
        └─ /receivers/
            └─ /R001/  attrs: position=[1000, 0]
            └─ /R002/  attrs: position=[2000, 0]
            └─ /R003/  attrs: position=[3000, 0]
```

対応 YAML:

```
sources:
  file: inputs/shot.h5          # 震源・受振器を含むv2.0ファイル
  shot_id: 0
receivers:
  type: [PS]                    # 全受振器の既定 type (@types で上書き可)
```


■例 2: 複数 shot・shot 内に複数 source.

inputs/multishot.h5

```

└ /shots/0000/                                attrs: shot_id=0
|   └ /sources/
|       └ /S0001/  attrs: id=1, type="force", direction=[1,0,0]
|           └ /stf
|       └ /S0002/  attrs: id=2, type="moment_tensor",
|                   moment_tensor=[Mxx,Myy,Mzz,Mxy,Mxz,Myz]
|           └ /stf
|   └ /receivers/
|       └ /R001/  attrs: position=[...], types=["DISP","VEL"]
|       └ /R002/  attrs: position=[...], types=["VEL"]
└ /shots/0001/                                attrs: shot_id=1
|   └ /sources/
|       └ /S0010/  attrs: id=10, type="pressure"
|           └ /stf
|   └ /receivers/
|       └ /R001/  attrs: position=[...], types=["DISP","VEL"]
|       └ /R002/  attrs: position=[...], types=["VEL"]
└ /shots/0002/                                attrs: shot_id=2
...

```

driver が shot_id=0,1,2,...を順次切り替えながら SEMSWS を起動する。各 shot 内の複数 sources は常に同時発火される。受振器ごとの @types 属性で、station 別に記録するチャンネルを変えられる。

4.14.3 出力経路: SEMSWS → HDF5

Self-roundtrip 仕様 と driver merge

■Self-roundtrip. 出力された各 seis<NNNN>.h5 は/sources/と/receivers/を両方含むため、そのまま SEMSWS の入力として再利用できる。例えば forward 結果を観測データとして次のインバージョンに渡す、forward 再現テストの参照ファイルとして使う、等。

semsws-driver を使う場合、driver が自動で per-shot directory layout を構築する:

```

output_root/
└ shot_0000/                                ← 入力 shot 0 の処理結果
|   └ config.yaml                            (shot 0 用に書き換えた YAML、shot_id=0)
|   └ seismograms0000.h5                    (内部 /shots/0000/、suffix も 0000)
|   └ stdout.log
└ shot_0001/                                ← 入力 shot 1
|   └ config.yaml                            (shot_id=1)

```

```

|   └─ seismograms0001.h5          (内部 /shots/0001/, suffix 0001)
|   └─ stdout.log
└─ shot_0002/
   └─ config.yaml                  (shot_id=2)
   └─ seismograms0002.h5          (内部 /shots/0002/, suffix 0002)
   └─ stdout.log

```

■driver の merge ツール. semsws-driver が複数 shot を順に走らせた場合、各 shot から生成される per-shot ファイル群を 1 つの HDF5 にまとめる post-process が用意されている (driver/tools/merge_shot_outputs.py 相当)。出力例:

```

shots_merged.h5
└─ /shots/
   └─ /0000/ (shot 0 の sources + receivers)
   └─ /0001/ (shot 1 の ...)
   └─ /0002/ (...)

```

各 per-shot ファイル中の /shots/<NNNN>/ がそのまま正しい index に入るため、merge ツールは shot_id の振り直しが不要で、単純に連結するだけで済む。

出力構成の例 (YAML 設定 → 生成 HDF5)

■例 1: 単震源 (典型的な forward シミュレーション) . YAML:

```

sources:
  shot_id: 0          # 省略すると 0
  list:
    - id: 1
      type: pressure
      location: [5000, 4000]
      wavelet: { type: ricker, frequency: 5.0, amplitude: 1.0, delay: 0.4 }
receivers:
  type: [PS]
  output:
    formats: [hdf5]
    filename: "seis"
  list:
    - { name: "R001", location: [1000, 0] }
    - { name: "R002", location: [2000, 0] }

```

生成ファイル (1 個) :

```

out/seis0000.h5          ← suffix=入力 shot_id=0

```

```

└ /shots/0000/                                attrs: shot_id=0
  └ /sources/
    └ /S0001/  attrs: id=1, type="pressure", position=[5000,4000]
      └ /stf                                (n_samples,)
    └ /receivers/
      └ /R001/  attrs: position=[1000,0]
        └ /PS                                (nt,)
      └ /R002/  attrs: position=[2000,0]
        └ /PS                                (nt,)

```

■例 2: 複数震源（同時発火、合成波形が 1 ファイル）. YAML:

```

sources:
  shot_id: 7                                # ← 入力 shot_id=7 と仮定
  list:
    - id: 1; type: force; location: [...]; direction: [1,0]; wavelet: {...}
    - id: 2; type: force; location: [...]; direction: [0,1]; wavelet: {...}
    - id: 3; type: pressure; location: [...]; wavelet: {...}
receivers:
  type: [DISP, VEL]
  output:
    formats: [hdf5]
    filename: "seis"
  list:
    - { name: "R001", location: [...] }

```

生成ファイル（HDF5 1 個に 3 source 合成結果が入る）:

```

out/seis0007.h5                                ← suffix=入力 shot_id=7
└ /shots/0007/                                attrs: shot_id=7
  └ /sources/                                  ← 全 active source のメタデータ
    └ /S0001/  (force, /stf)
    └ /S0002/  (force, /stf)
    └ /S0003/  (pressure, /stf)
  └ /receivers/
    └ /R001/                                  ← 3 source 同時発火の合成波形
      └ /DISP                                (space_dim, nt)
      └ /VEL                                (space_dim, nt)

```

重要: 3 source あっても HDF5 ファイルは 1 個。各受振器の波形データセットには 3 source 合成結果が入る。ファイル名 suffix・内部/shots/<NNNN>/ 共に入力 shot_id を反映。

第 5 章

ツールとスクリプト

SEMSWS にはシミュレーション本体 (semsws) に加え、メッシュの分割・細分化・品質評価などの前処理ツールと、結果の可視化用 Python スクリプトが付属する。本章ではこれらの使い方を説明する。

5.1 partition_mesh — メッシュの事前分割

partition_mesh は、メッシュファイルを MPI 並列実行用に事前分割するツールである (serial run only)。分割済みメッシュを使うことで、シミュレーション起動時のメッシュの読み込み・分割のオーバーヘッドを省略できる。同じメッシュ・分割数でシミュレーションを複数回実行する場合に有用である。

5.1.1 使い方

```
# 外部メッシュを METIS で 64 分割
./build/src/partition_mesh -n 64 domain.exo partitions/

# Cartesian 分割 (4*4*4 = 64)
./build/src/partition_mesh -n 64 -p cartesian -g 4,4,4 domain.exo partitions
/

# 内部メッシュを生成して分割
./build/src/partition_mesh -n 64 --internal \
  --size 10000,10000,5000 --elements 100,100,50 partitions/
```

5.1.2 オプション

オプション	説明
<code>-n <nparts></code>	分割数（必須）
<code>-p <method></code>	分割方法： <code>metis</code> （デフォルト）または <code>cartesian</code>
<code>-g <nx,ny,nz></code>	Cartesian 分割のグリッド数
<code>--internal</code>	内部メッシュ生成モード
<code>--dim <2 3></code>	次元（内部メッシュ時、デフォルト：3）
<code>--origin <x,y,z></code>	原点（内部メッシュ時）
<code>--size <lx,ly,lz></code>	ドメインサイズ（内部メッシュ時、必須）
<code>--elements <nx,ny,nz></code>	要素数（内部メッシュ時、必須）

出力ファイルは指定ディレクトリに `mesh.mesh.000000`, `mesh.mesh.000001`, ... として生成される。

5.1.3 config.yamll での使用

分割済みメッシュをシミュレーションで使用するには、`config.yaml` の `mesh` セクションを以下のように設定する：

```
mesh:
  type: prepart_mfem
  prepart_mfem:
    directory: "partitions/"
    nparts: 64
```

`nparts` は MPI 実行時のプロセス数と一致させる必要がある。

出力ディレクトリには `mesh.mesh.000000` 等のパーティションファイルに加えて `partition_info.yaml`（分割数・要素数などのメタデータ。読み込みには不要）も生成される。

5.2 combine_mesh — 分割メッシュの結合

`combine_mesh` は、`partition_mesh` で分割したメッシュを 1 つのファイルに結合するツールである。GLVis での可視化や、分割メッシュの確認に使用する。

```
# 64分割のメッシュを結合（分割数と同じnpで実行）
mpirun -np 64 ./build/src/combine_mesh partitions/ combined.mesh
```

実行時の MPI プロセス数は分割数と一致させる必要がある。

結合されたメッシュは GLVis で確認できる：

```
glvis -m combined.mesh
```

5.3 evaluate_mesh — メッシュ品質の評価

evaluate_mesh は、メッシュの品質を評価し、シミュレーションパラメータとの整合性を確認するツールである。要素サイズ、CFL 条件、PPW (Points Per Wavelength)、アスペクト比、ヤコビアン行列式の統計量を算出する。

5.3.1 使い方

```
# 基本的な評価（コンソール出力のみ）
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml

# ヒストグラム CSV ファイルも出力
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist --nbins 50
```

5.3.2 オプション

オプション	説明
<code>--config <file></code>	設定ファイル（必須）
<code>--cfl <factor></code>	CFL 係数のオーバーライド
<code>--histogram <prefix></code>	ヒストグラム CSV の出力接頭辞
<code>--nbins <N></code>	ヒストグラムのビン数（デフォルト：50）

5.3.3 出力されるヒストグラム

`--histogram` を指定すると、以下の CSV ファイルが生成される：

ファイル	内容
<prefix>_element_size.csv	要素サイズ分布
<prefix>_dt_cfl.csv	CFL 条件による $\Delta t_{\max}$ 分布
<prefix>_ppw.csv	PPW 分布
<prefix>_aspect_ratio.csv	アスペクト比分布
<prefix>_jacobian_det.csv	ヤコビアン行列式分布

5.4 refine_mesh — メッシュの均一細分化と Partitioning

`refine_mesh` は、物性値に基づいてメッシュを均一に細分化するツールである。低速度域（波長が短い領域）の要素を自動的に分割し、PPW 条件を満たすメッシュを生成する。

細分化の基準は以下の式で決まる。まず、必要な要素サイズ h_{required} を求める：

$$h_{\text{required}} = \frac{v_{\min} \times N_{\text{GLL}}}{f_{\max} \times \text{PPW}}$$

ここで $v_{\min}$ は全要素中の最小波速度、 $N_{\text{GLL}} = \text{order} + 1$ 、 $f_{\max}$ と PPW は config の `mesh.max_freq` と `mesh.ppw` で指定する。

次に、現在の要素サイズ h_{current} から細分化回数 n を決定する：

$$n = \left\lceil \log_2 \frac{h_{\text{current}}}{h_{\text{required}}} \right\rceil$$

$h_{\text{current}} \leq h_{\text{required}}$ であれば細分化は不要 ($n = 0$)。全要素の最大 n がメッシュ全体の細分化レベルとなる。

注意として、メッシュ全体を均一に細分化する。1 回の細分化で各要素が 2 分割されるため、2 次元では要素数が 2^{2n} 倍、3 次元では 2^{3n} 倍になる。そのため、PPW などの指標はある程度の妥協が必要になる場合がある。

主な用途は二つある。一つは PPW 条件を満たすメッシュの作成、もう一つは HPC での大規模計算（大規模メッシュ）への対応である。**SEMSWS では、`mesh.type: prepart_mfem` を指定しない限り、並列計算であっても Rank 0 がメッシュ全体を読み込んでから分割し、各ランクへ `MPI_Send` で配布する。そのため大規模メッシュではメモリに収まらない問題が生じる。また、CUBIT などで事前に分割（partitioning）したメッシュは読み込めず、`partition_mesh` による事前分割も最初は Rank 0 が全メッシュを読み込んでしまう。**このような場合には、地形（topography）や海底地形（bathymetry）といった概略の形状を捉えられる粗いメッシュを CUBIT や GMSH で作成し、`refine_mesh` で細分化するという方法がある。`refine_mesh` は後述のとおり、入力メッシュを Rank 0 で読み込み、メッシュを分割し、各ランクで細分化したうえで、分割済みメッシュを出力す

る。出力されたメッシュは、`mesh.type: prepart_mfem` を `config` に設定すれば、各ランクが自分のパーティションのメッシュのみを読み込む。

5.4.1 メッシュの読み込みと出力の挙動

`refine_mesh` は内部で `LoadParMesh()` を呼び出してメッシュを読み込む。読み込みパスは `config` の `mesh.type` によって異なる：

■未分割メッシュ (`type: internal / external`) の場合 Rank 0 のみがシリアルメッシュを読み込み (`internal` の場合は生成し)、`MeshPartitioner` で N 分割した後、各ランクに `MPI_Send` で配布する。 N は `mpirun -np N` のプロセス数で決まる。

■分割済みメッシュ (`type: prepart_mfem`) の場合 各ランクが自分のパーティションファイル (`mesh.mesh.NNNNNN`) を独立に読み込む。実行時の MPI プロセス数と `nparts` が一致している必要がある。

■リファインと出力 どちらのケースでもリファイン自体は `ParMesh::UniformRefinement()` で並列に動作し、挙動に違いはない。違うのは読み込みパスのみである。リファイン後の出力は 2 通り：

- `--output-dir`: 各ランクが `ParPrint()` で自分のパーティションファイルを書き出す (N 個のファイル)。シミュレーションで `type: prepart_mfem` として使用する
- `--save-as-one`: Rank 0 が `SaveAsOne()` で 1 つのファイルに結合する。GLVis での可視化やシミュレーションで `type: external` として使用する

5.4.2 使い方

```
# 8プロセスで実行 (MPI必須)
mpirun -np 8 ./build/src/refine_mesh --config config.yaml --output-dir MESH/

# 可視化用の結合メッシュも同時に保存
mpirun -np 8 ./build/src/refine_mesh --config config.yaml \
  --output-dir MESH/ --save-as-one refined.mesh
```

5.4.3 オプション

オプション	説明
<code>--config <file></code>	設定ファイル (必須)
<code>--output-dir <dir></code>	分割メッシュの出力先 (必須)
<code>-n <max_ref></code>	最大細分化レベルの上書き
<code>--save-as-one <file></code>	結合メッシュも保存 (GLVis 可視化用)

出力は `partition_mesh` と同じ形式 (`mesh.mesh.NNNNNN` と `partition_info.yaml`) であり、`config.yaml` で `type: prepart_mfem` として直接使用できる。

5.5 ワークフロー例：メッシュ評価と細分化

ここでは `examples/visco_elastic_3d_layered/` の 2 層粘弾性モデルを例に、メッシュの評価から細分化、再評価までの一連のワークフローを示す。

5.5.1 問題設定

10 km × 10 km × 10 km の 3 次元ドメインに 2 層の粘弾性媒質を設定する。上層 ($z > -4000$ m、attribute 2) は低速度 ($V_p = 3000$ m/s, $Q_\kappa = Q_\mu = 50$)、下層 ($z \leq -4000$ m、attribute 1) は高速度 ($V_p = 6000$ m/s, $Q_\kappa = Q_\mu = 100$) である (この attribute 番号は物性ファイルおよび評価出力の Attr 1/2 と対応する)。

メッシュは CUBIT (無料版 Coreform で可) で生成する。付属のジャーナルファイル `mesh_layered_3d.jou` を実行すると ExodusII 形式のメッシュが生成される：

```
# CUBIT GUI CONSOLEで以下を実行
playback 'mesh_layered_3d.jou'
```

物性ファイル `materials_visco.txt` の内容：

#	attribute	v_p	v_s	ρ	q_κ	q_μ
1		6000.0	3500.0	2700.0	100.0	100.0
2		3000.0	2000.0	2200.0	50.0	50.0

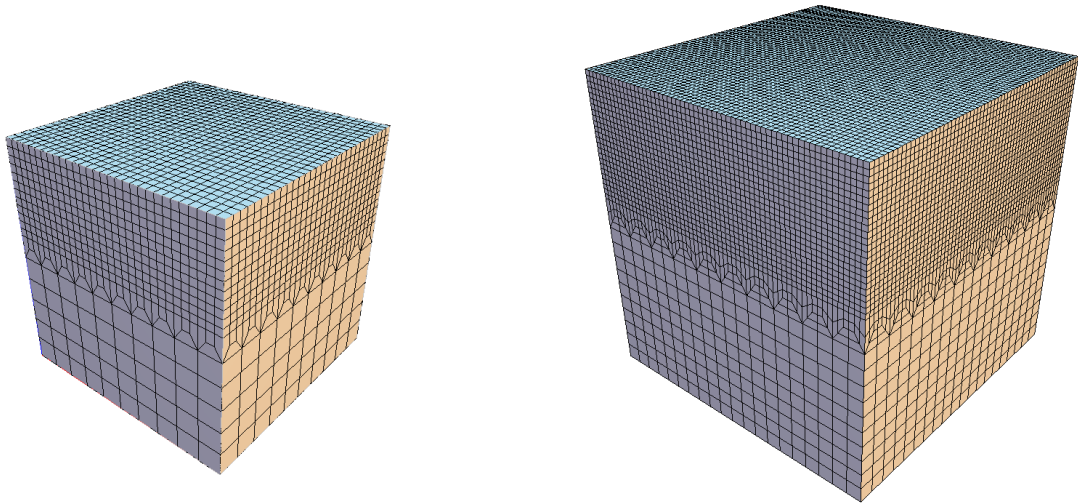


図 5.1 細分化前 (左) と細分化後 (右) のメッシュ。左: CUBIT で生成した初期メッシュ (12,600 要素)。上層は 500 m、下層は 1,000 m 要素。右: `refine_mesh` で 1 回均一細分化後 (100,800 要素)。全要素が $2^3 = 8$ 倍に分割されている。

5.5.2 Step 1: 初期メッシュの評価

まず初期メッシュの品質を評価する：

```
./build/src/evaluate_mesh --config config_visco.yaml --histogram mesh
```

出力例：

```
Mesh Evaluation Report
=====
Mesh: 12600 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
```

	min	max	mean	std
h [m]	3.333e+02	1.000e+03	3.858e+02	1.561e+02
dt_CFL [s]	2.141e-03	8.562e-03	5.394e-03	1.227e-03
PPW	3.163e+00	9.490e+00	5.436e+00	7.247e-01
Aspect ratio	1.000e+00	8.492e+00	1.331e+00	1.191e+00
det(J)	2.778e+07	1.000e+09	7.540e+07	1.883e+08

```

CFL summary:
  dt_max (global): 2.1406e-03 s

PPW summary:
  Worst PPW: 3.16 at centroid (9500.00, 4500.00, -6500.00)

```

```
PPW < 5.0: 1300 / 12600 (10.32%)
```

Per-attribute breakdown:

```
Attr 1: 1800 elems, Vp=6050, Vs=3543, PPW_min=3.2, dt_min=2.14e-03
```

```
Attr 2: 10800 elems, Vp=3066, Vs=2049, PPW_min=5.5, dt_min=5.63e-03
```

PPW の最小値が 3.16 と目標の 5 を下回っており、Attr 1 (下層、高速度域) の 1300 要素 (10.3%) で PPW 不足であることがわかる。

5.5.3 Step 2: 均一細分化

`refine_mesh` でメッシュを細分化する。ここでは 4 プロセスで実行するが、付属の `run_visco.sh` では 64 プロセスで実行している：

```
mpirun -np 4 ./build/src/refine_mesh \
  --config config_visco.yaml \
  --output-dir mesh/
```

出力例：

```
Loading mesh...
  Elements (global): 12600

Loading material...
  V_min (global): 2049.22
  V_max (global): 6050.01

Refinement levels needed: 1
  Refinement 1: 100800 elements (global)

Refined mesh:
  Elements (global): 100800
  h range:           [84.12, 500.03]
  PPW (worst):       6.33 (target: 5)
  CFL dt_max:        0.00107 s (CFL=0.3)

Saving partitioned mesh to: mesh/
  4 partition files (mesh.mesh.NNNNNN)
```

1 回の細分化で 12,600 要素が 8 倍の 100,800 要素となり、PPW 最小値が 3.16 から 6.33 に改善された (図 5.1)。

5.5.4 Step 3: 細分化後の再評価

細分化後のメッシュを再評価し、PPW 条件が満たされたことを確認する。`config_visco_sim.yaml` では `mesh.type: prepart_mfem` で細分化済みメッシュを参照する：

```
mpirun -np 4 ./build/src/evaluate_mesh --config config_visco_sim.yaml
```

出力例：

```
Mesh Evaluation Report
=====
Mesh: 100800 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
```

	min	max	mean	std
h [m]	1.667e+02	5.000e+02	1.938e+02	8.145e+01
dt_CFL [s]	1.070e-03	4.281e-03	2.718e-03	5.622e-04
PPW	6.326e+00	1.898e+01	1.087e+01	1.563e+00
Aspect ratio	1.000e+00	8.495e+00	1.230e+00	8.541e-01

```

PPW summary:
  Worst PPW: 6.33 at centroid (250.00, 9250.00, -9250.00)
  PPW < 5.0: 0 / 100800 (0.00%)

Per-attribute breakdown:
  Attr 1: 14400 elems, Vp=6050, Vs=3543, PPW_min=6.3, dt_min=1.07e-03
  Attr 2: 86400 elems, Vp=3066, Vs=2049, PPW_min=11.0, dt_min=2.82e-03
```

PPW 不足の要素が 0% となり、全要素で $PPW \geq 5$ の条件を満たしている。

5.5.5 Step 4: シミュレーションの実行

細分化済みメッシュでシミュレーションを実行する。`type: prepart_mfem` では、MPI プロセス数を Step 2 で作成したパーティション数（ここでは 4）と一致させる必要がある（`run_visco.sh` のように 64 プロセスで細分化した場合は、シミュレーションも `-np 64` で実行する）：

```
mpirun -np 4 ./build/src/semsws \
  --config config_visco_sim.yaml
```

5.6 可視化スクリプト

scripts/plot/に Python による可視化スクリプトが用意されている。いずれも matplotlib と numpy が必要である。

5.6.1 plot_wavefield_2d.py — 2D 波動場の可視化

GMT 形式で出力された 2D 波動場スナップショットと物性分布を matplotlib で可視化する。

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

results/figures/に以下のファイルが生成される：

- wavefield_snapshots.png — 波動場スナップショット（全タイムステップ）
- material.png — 物性分布

出力例は第 3 章の図 3.1 を参照。

5.6.2 plot_wavefield_3d.py — 3D 断面波動場の可視化

GMT 形式の 3D 断面（cross_sections）を matplotlib で可視化する。

```
python3 scripts/plot/plot_wavefield_3d.py --results-dir ./results
```

config で指定した各断面（yz, xz, xy）ごとにスナップショットが生成される。図 5.2 に 2 層粘弾性モデルの xz 断面（ $y = 5000$ m）の出力例を示す。

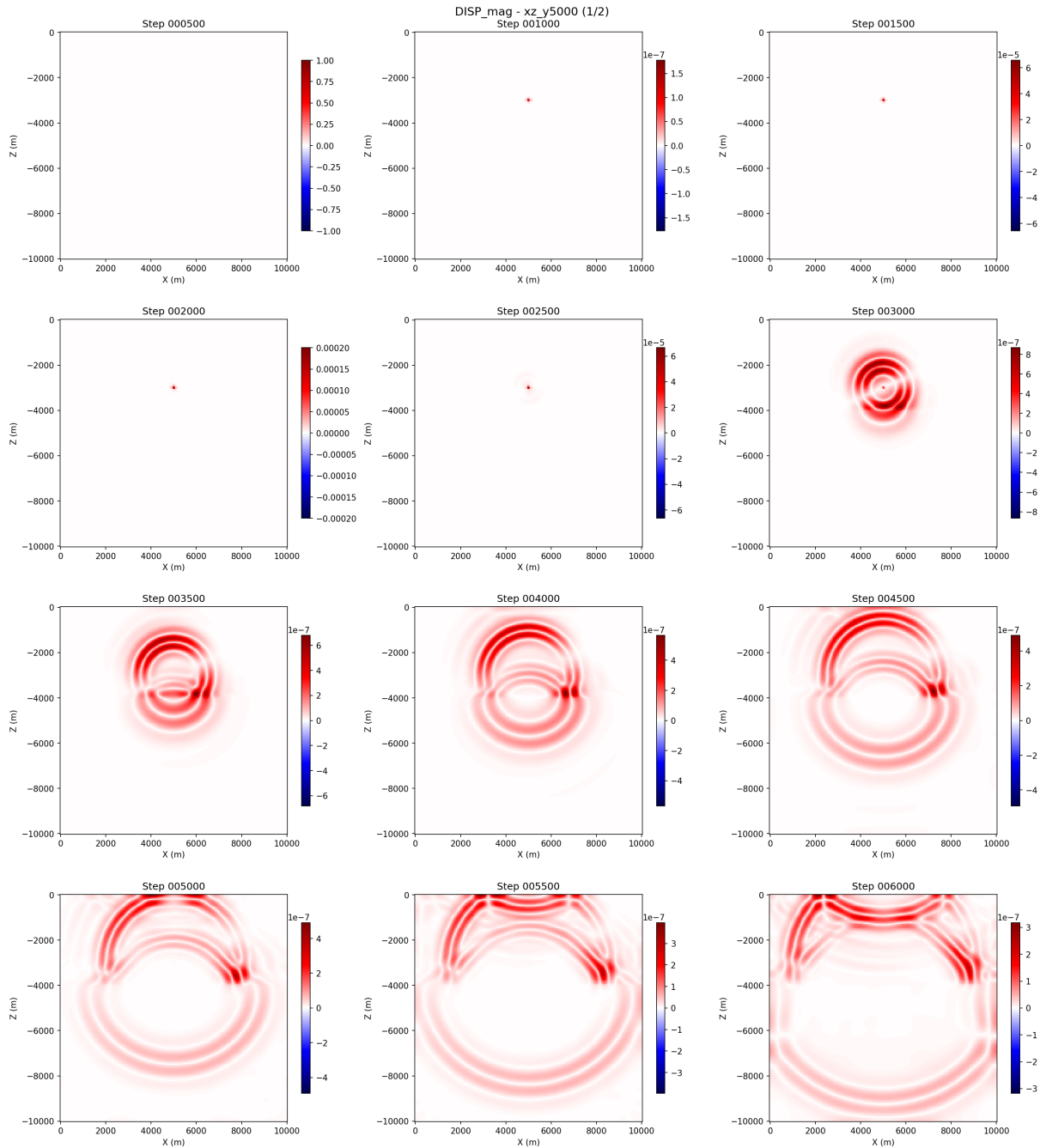


図 5.2 `plot_wavefield_3d.py` による 3D 断面スナップショット (xz 断面、 $y = 5000\text{m}$)。2 層粘弾性モデルでの変位マグニチュード。500 ステップ間隔で出力。

5.6.3 `plot_3d_slices.py` — PyVista 3D 可視化

PyVista を使用して、3 つの断面を 3D 空間上に配置した可視化を行う。`pyvista` パッケージが追加で必要である。

```
python3 scripts/plot/plot_3d_slices.py --results-dir ./results \
--type wavefield --step 6000
```

オプション	説明
<code>--results-dir</code>	結果ディレクトリ
<code>--type</code>	wavefield / material / both (既定)
<code>--step</code>	表示するタイムステップ番号 (wavefield 時)

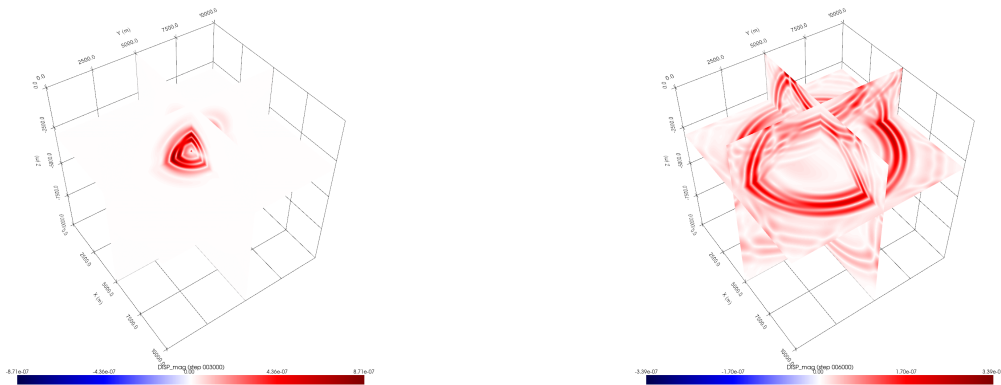


図 5.3 `plot_3d_slices.py` による PyVista 3D 可視化。yz, xz, xy 断面を 3D 空間上に配置。
左 : Step 3000、右 : Step 6000。2 層粘弾性モデルの変位マグニチュード。

5.6.4 `plot_mesh_quality.py` — メッシュ品質ヒストグラム

`evaluate_mesh` が出力した CSV ファイルからヒストグラム図を生成する。

```
# evaluate_meshでCSV出力
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
    --histogram mesh_hist

# ヒストグラム図を生成
python3 scripts/plot/plot_mesh_quality.py mesh_hist
python3 scripts/plot/plot_mesh_quality.py mesh_hist --output quality.png
```

要素サイズ、CFL $\Delta t_{\max}$ 、PPW、アスペクト比、ヤコビアン行列式の各ヒストグラムが 1 枚の図にまとめられる。--dpi で出力解像度を指定できる (既定 150)。

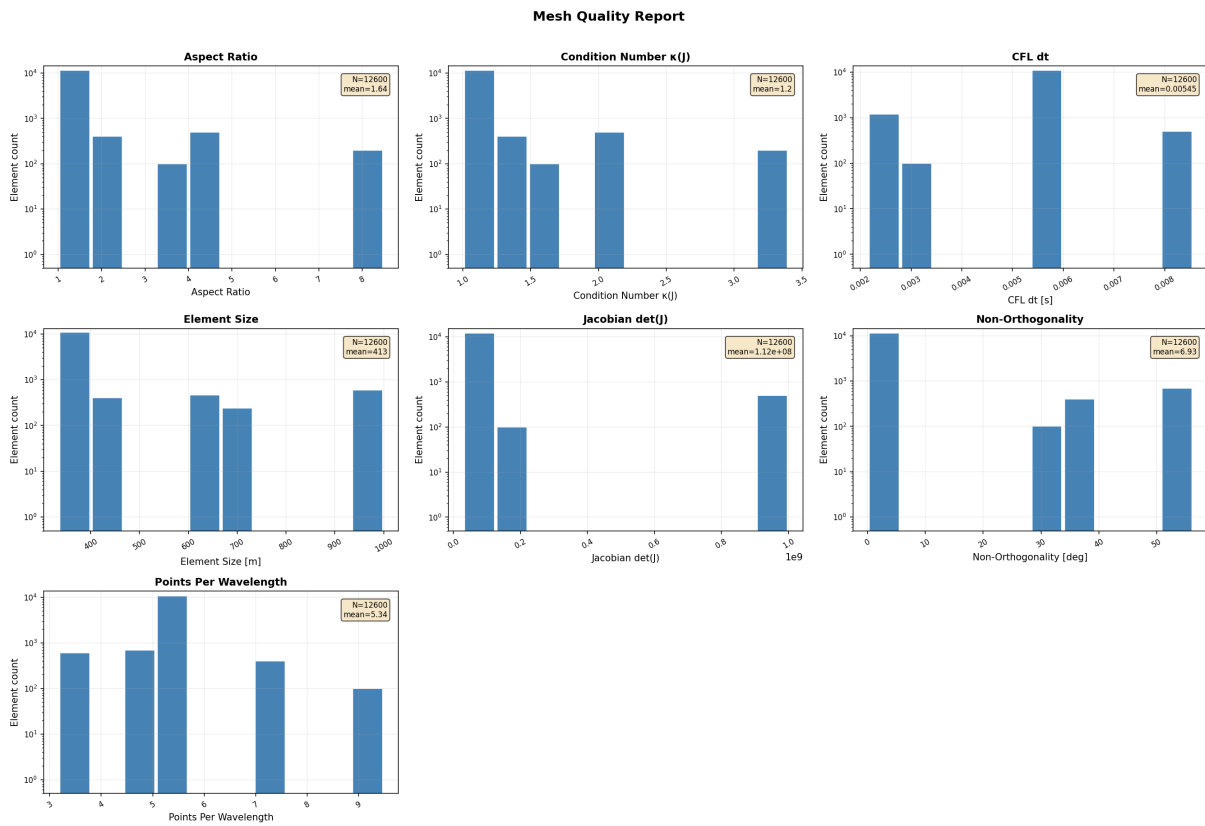


図 5.4 メッシュ品質ヒストグラム（細分化前の初期メッシュ）。`evaluate_mesh` の出力を `plot_mesh_quality.py` で可視化。

付録 A

サイト別ビルド設定

本付録では HPC サイト固有の手動ビルド手順と、Spack を使う場合の spec 例をまとめる。本文 §2.3・§2.2 と合わせて参照すること。

A.1 Earth Simulator (NVIDIA A100 GPU)

JAMSTEC の Earth Simulator の GPU パーティション (NVIDIA A100) 向けの手順を示す。2025 年 4 月時点で動作確認した構成は次のとおり。NVIDIA HPC SDK 24.7、CUDA 12.0.0、hypre 2.31.0-2.33、METIS 5.1.0、HDF5 1.10.5。

A.1.1 手動ビルド

```
# モジュールのロード
module load HDF5/1.10.5
module load NetCDF4/all
module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12
module load METIS/5.1.0
module load CUDA/12.0.0

# hypre
cd $HOME/program_gpu/hypre/src
./configure \
  --enable-single --enable-mixedint --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/opt/share/CUDA/12.0.0 HYPRE_CUDA_SM=80
make -j$(nproc)

# MFEM
cd $HOME/program_gpu/mfem
```

```

cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DMFEM_USE_MPI=ON -DMFEM_USE_CUDA=ON \
  -DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$HOME/program_gpu/hypre/src/hypre \
  -DMETIS_DIR=/opt/share/METIS/5.1.0
cmake --build build -j$(nproc)

# SEMSWs
cd $HOME/program_gpu/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DSEM_USE_GPU_AWARE_MPI=ON \
  -DMFEM_DIR=$HOME/program_gpu/mfem/build \
  -Dadios2_DIR=$HOME/program_cpu/ADIOS2/build
cmake --build build -j
cmake --install build

```

A.1.2 Spack

ES ではコンパイラ・MPI・CUDA の external 登録と、spec への伝播フラグの指定が必要となる。spack-repo/site-config/es/にロードスクリプトと packages.yaml テンプレートが用意されている。

■GPU env

```

export WORK=/S/data01/G3506/${USER}/program_test
git clone --depth=1 https://github.com/spack/spack.git $WORK/spack
cd $WORK && git clone https://github.com/SEMSWS/semsws.git && cd semsws

cp spack-repo/site-config/es/load_gpu.sh ~/load_gpu.sh
chmod +x ~/load_gpu.sh
source ~/load_gpu.sh

```

```
spack compiler find /usr/bin
spack repo add $WORK/semsws/spack-repo
spack env create semsws-gpu && spack env activate semsws-gpu
spack config edit          # packages_gpu.yaml と下記 specs を貼り付け
```

spack config edit で開いた spack.yaml に spack-repo/site-config/es/packages_gpu.yaml の内容を貼り、specs を追加する：

```
spack:
  repos:
    - /S/data01/G3506/<USER>/program_test/semsws/spack-repo
  packages:
    # packages_gpu.yaml の中身をここに貼る
  specs:
    - semsws@main +cuda cuda_arch=80 precision=single +gpu_aware_mpi
      cxxflags=="-noswitcherror" cflags=="-noswitcherror"
      ldlibs=="-lstdc++fs"
      ^hydre@2.33
      %nvhpc
  concretizer:
    unify: true
  config:
    build_jobs: 4
```

その後：

```
spack concretize -f
spack install --fail-fast 2>&1 | tee install_gpu.log
```

初回フルビルドは 50-90 分 (hydre CUDA kernel の nvcc compile が 30 分以上)。

■CPU env GPU 版とは別 env として管理する。NVIDIA HPC SDK とシステムの OpenMPI は同時にロードできないため、load_gpu.sh と load_cpu.sh のどちらか一方だけを読み込むように切り替える：

```
spack env deactivate
source ~/load_cpu.sh
spack env create semsws-cpu && spack env activate semsws-cpu
spack config edit          # packages_cpu.yaml を貼り、specs:
                          # - semsws@main precision=single ^hydre@2.33 %gcc
spack concretize -f
spack install --fail-fast 2>&1 | tee install_cpu.log
```

A.1.3 2 回目以降のセッション

```
# GPU版
source ~/load_gpu.sh && spack env activate semsws-gpu && spack load semsws

# CPU版
source ~/load_cpu.sh && spack env activate semsws-cpu && spack load semsws
```

`source` が必要なのは GPU と CPU で system module がどちらか一方しか使えないため (Spack env の activate だけでは module 環境は切り替わらない)。

A.1.4 PJM ジョブスクリプト例

```
#!/bin/bash
#PJM -L rscgrp=gpu
#PJM -L node=1
#PJM -L elapse=01:00:00
#PJM -j

source ~/load_gpu.sh
spack env activate semsws-gpu
spack load semsws

# 1ノード上で 8 ranks (A100 x 8 にそれぞれバインド)
mpirun -np 8 -map-by ppr:8:node \
    -x PATH -x LD_LIBRARY_PATH \
    semsws --config config.yaml
```

複数ノードでは `-x PATH -x LD_LIBRARY_PATH` を必ず付与する。`spack load` は head ノードのシェルにしか `PATH/LD_LIBRARY_PATH` を設定しないため、これを忘れるとリモートノードで `semsws: command not found` や共有ライブラリの `cannot open shared object file` が発生する。

A.2 LUMI (AMD MI250X GPU)

CSC Finland の LUMI スーパーコンピュータ (LUMI-G パーティション) 向けの手順を示す。2026 年 4 月時点で動作確認した構成は次のとおり。ROCm 6.4.4、Cray PE 25.09 (CCE 20.0.0)、hydre 3.1.0、METIS 5.1.0 (cpeCray-25.09-idx32-fp32)、cray-hdf5 1.14.3.7。

A.2.1 手動ビルド

```

# モジュールのロード
module load LUMI/25.09 partition/G
module load METIS/5.1.0-cpeCray-25.09-idx32-fp32
module load cray-hdf5/1.14.3.7

# hypre
cd $FD/program/hypre/src
./configure \
  --enable-single --enable-mixedint --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc \
  LDFLAGS="-L/opt/cray/pe/mpich/9.0.1/gtl/lib" \
  LIBS="-lmpi_gtl_hsa"
make -j$(nproc)

# MFEM
cd $FD/program/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DMFEM_USE_MPI=ON -DMFEM_USE_HIP=ON \
  -DAMDGPU_TARGETS=gfx90a \
  -DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$FD/program/hypre/src/hypre \
  -DHDF5_INCLUDE_DIRS=/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/include \
  -DHDF5_LIBRARIES="/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5_cpp.so
    ;\
/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5.so"
cmake --build build -j$(nproc)

# SEMSW
cd $FD/program/SEMSW
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=<install_dir> \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DAMDGPU_TARGETS=gfx90a \

```

```
-DSEM_USE_GPU_AWARE_MPI=ON \  
-DMFEM_DIR=$FD/program/mfem/build \  
-Ddaios2_DIR=$FD/program/ADIOS2/build  
cmake --build build -j  
cmake --install build
```

MPI ジョブの投入には `srun` を使用する。

開発チーム

- Kota Mukumoto — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain
- Yann Capdeville — Nantes Université, Univ Angers, Le Mans Université, CNRS, Laboratoire de Planétologie et Géosciences, Nantes, France
- Kazuya Shiraishi — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Ryoichiro Agata — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Gou Fujie — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Thomas Bodin — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain